

Terracotta: Enabling the Adoption of New DRAM Techniques via a Flexible DRAM Interface and Memory Controller

Harsh Songara¹ Konstantinos Kanellopoulos¹ F. Nisa Bostancı¹ Konstantinos Marios Sgouras¹
Ataberk Olgun¹ İsmail Emir Yüksel¹ Andreas Kosmas Kakolyris¹ A. Giray Yağlıkçı² Onur Mutlu^{1,3}

¹ETH Zürich ²CISPA ³New York University

DRAM is the dominant memory technology in modern systems, yet it continues to limit performance, energy efficiency, and system robustness. To mitigate such limitations, many prior works have proposed novel DRAM techniques that change how DRAM is operated to support in-DRAM computation, improve memory access latency and parallelism, and enhance DRAM maintenance and reliability. However, adopting these techniques one after another in current systems with rigid interfaces and memory controller hardware requires repeated and costly modifications to them, hindering the deployment of critical performance, energy efficiency, and robustness improvements.

Our goal is to reduce the repeated DRAM interface and memory controller modifications that hinder or slow down the deployment of diverse DRAM techniques. We observe that the DRAM commands and memory controller structures across several DRAM techniques are similar. Our key idea is to leverage these similarities to compose a set of primitives that DRAM vendors and system designers can use to implement a wide variety of DRAM techniques. To this end, we propose Terracotta, a new framework that consists of two flexible components: (i) custom command extensions that enable DRAM vendors to define new commands within a single, standardized interface, and (ii) a programmable memory controller that system designers can easily and flexibly program to support new DRAM techniques post-silicon fabrication. Together, these two components enable the deployment of new DRAM techniques via configuration of the memory controller rather than repeated interface and controller modifications.

We design Terracotta for a DDR5-based system and comprehensively evaluate its performance, energy, and hardware complexity. We draw three key findings. First, for four DRAM techniques from four distinct domains including processing-using-DRAM, low-cost DRAM maintenance, subarray-level parallelism, and latency reduction, Terracotta retains almost all of the performance benefits (>96%) that custom implementations provide over a baseline system without these techniques. Second, a Terracotta-based composition of two DRAM techniques from the domains of subarray-level parallelism and latency reduction outperforms the Terracotta-based implementations of the individual techniques, showing that Terracotta can enable system improvements by adding techniques without further interface and controller modifications. Third, Terracotta’s flexibility incurs low DRAM energy overheads (0.6–3.2%) and low area and power overheads (0.03% and 0.56%, respectively) in a high-end server-grade processor. Terracotta’s source code is freely available at <https://github.com/CMU-SAFARI/Terracotta>.

1. Introduction

Dynamic Random Access Memory (DRAM) [1,2] is the dominant main memory technology across a broad range of modern computing systems because of its high capacity and relatively low latency at low cost [3–7]. However, DRAM continues to limit system performance, energy efficiency, and robustness [3–51], a problem greatly exacerbated today due to the memory demand of emerging Artificial Intelligence (AI) and Machine Learning (ML) workloads [52–68]. To mitigate these limitations, many prior works propose *DRAM techniques* that change how DRAM is architected, operated, and managed in various novel ways [8–12,34–48,69–299]. These techniques (i) leverage DRAM’s analog processing capability and massive parallelism across columns to enable processing-using-DRAM (PuD) [8–12,34,35,69–114], (ii) enable low-cost maintenance operations (e.g., read disturbance mitigation, refresh, memory scrubbing, and online error profiling) [36–47,115–275], (iii) expose DRAM’s parallelism across subarrays to the host system [48,230–232,276,277], and (iv) reduce DRAM access latency [220,278–295].

Motivation. DRAM techniques typically require modifications to two parts of the system: (i) the DRAM interface [182] and (ii) the memory controller [4,300–306]. First, several DRAM techniques expose novel DRAM functionalities to the memory controller via new, non-standard DRAM commands [34,48,104–110,217–221,232–235,276,278,290–299], requiring modifications to the DRAM interface. Second, several DRAM techniques require memory controller hardware support to determine when to invoke and how to perform a DRAM technique [34–39,44,45,48,69–110,182–258,270–289,292–299].

The DRAM interface and controller modifications required by many DRAM techniques pose two deployment obstacles. First, implementing a DRAM technique that needs DRAM interface and memory controller support may require the development of a new DRAM standard [307–323]. This can be prohibitive because it requires agreement among all the stakeholders of the DRAM standard, such as DRAM vendors and system designers. These stakeholders may have conflicting goals, approaches, and interests [5,6,181,182], potentially hindering or slowing down the adoption of techniques that benefit a subset of stakeholders. Second, even if the stakeholders agree to support a DRAM technique in future computing systems, already-deployed systems *cannot* easily adopt it. Such systems remain in the field for several years [216,324–326] (or even decades for satellite [327,328] and space systems [329–339]). During this time, new DRAM techniques may become valuable

and useful to help address the changing memory demands of various workloads [18,29,33,52,58–61,73,340,341], including fast-evolving ones like AI and ML. Together, these two obstacles hinder the adoption of new DRAM techniques. Both obstacles stem from the rigidity of current DRAM interfaces and memory controllers, whose supported commands and control logic are fixed at design time. Adopting techniques that require new commands and controller support therefore entails repeated and costly interface and controller modifications, depriving computing systems of performance and energy efficiency improvements and essential security features [6,40,43,182,215,216,342].

Our goal is to reduce the repeated interface and controller modifications that hinder or slow down the deployment of diverse DRAM techniques. Our analysis of 32 DRAM techniques [34–36,48,80–83,104,220–231,278–282,294–298] across seven different categories reveals that these techniques rely on similar command structures and memory controller workflows. Our **key idea** is to leverage these similarities to compose a set of *primitives* that DRAM vendors and system designers can independently use to implement a diverse set of DRAM techniques. To this end, we propose **Terracotta**, a new memory control framework that consists of two flexible components: (i) *custom command extensions* that enable DRAM vendors to independently define new commands within a single standardized interface extension, requiring *only* a one-time interface change instead of repeated DRAM interface changes, and (ii) a *programmable memory controller* that enables system designers to express memory controller support for new DRAM techniques post-silicon fabrication.

Custom command extensions. We observe that DRAM commands of various techniques have a common structure. We illustrate this using two examples. First, PRADA [104,105] introduces a partial row activation command, A^* , that activates a row without performing sense amplification. Second, CROW [220] introduces a row copy command, ACT-c, that copies data from a source row to a destination row in DRAM. Both A^* and ACT-c perform a DRAM operation in a specific DRAM locality (e.g., a specific bank) and operate on one or more operands (e.g., row addresses). Terracotta leverages the common structure in such commands to define three primitives: (i) *operation type* (e.g., partial row activation or row copy), (ii) *target DRAM locality* (e.g., a specific bank), and (iii) *payload* (e.g., row addresses). Terracotta uses these primitives to extend the DRAM interface through a one-time change that standardizes an encoding for custom commands. This encoding *only* defines the structural format (i.e., bit positions and widths) as a fixed agreement between system designers and DRAM vendors, separating structure from the functional meaning of each command. Terracotta’s custom command extensions allow DRAM vendors to expose new DRAM functionalities autonomously, without further interface changes or explicit agreement among the stakeholders of the DRAM standard.

Programmable memory controller. Applying the same approach to the memory controller, we identify common controller workflows across different DRAM techniques. For in-

stance, two existing DRAM techniques, ChargeCache [279] and CROW [220], reduce row activation latency by leveraging high DRAM cell charge in recently precharged and copied rows, respectively. Despite targeting different row types, both techniques require the memory controller to track eligible rows, check incoming requests against these rows, and perform reduced-latency operations when an incoming request targets an eligible row. Terracotta decomposes this common structure into four primitives: *state*, *triggers*, *updates*, and *actions*. First, *state* is the microarchitectural information the controller maintains for the operation of a DRAM technique (e.g., recently accessed rows). Second, a *trigger* fires a DRAM technique when a *condition* over the state and an incoming memory request is met (e.g., a read request targets a tracked row). Third, an *update* maintains the state based on the memory requests issued by the memory controller (e.g., inserting a row into a tracking table on an activation). Fourth, an *action* issues custom DRAM commands, if a technique needs them, by leveraging Terracotta’s command extensions. Terracotta exposes programmatic control over these primitives, allowing system designers to implement new DRAM techniques post-silicon fabrication without modifying the memory controller.

Key benefits. Terracotta’s flexible framework allows both DRAM vendors and system designers to enjoy improvements in autonomy and collaboration in four ways. First, Terracotta enables DRAM vendors to unilaterally implement DRAM techniques that they (or their collaborators) see fit, *without* requiring approval from *all* the stakeholders of the DRAM standard. Second, Terracotta allows DRAM vendors and system designers to collaborate beyond the DRAM standard to come up with much-needed innovative solutions [5,6,8–11,40,181] to mitigate the limitations of modern DRAM-based systems. Third, both DRAM vendors and system designers can continue to maintain the proprietary nature of their designs. Fourth, system designers can freely choose among many DRAM chips that can implement different DRAM techniques, as performance, energy efficiency, and security needs change, enabling differentiation across different vendors. In turn, Terracotta’s flexibility can foster a more innovative ecosystem, enabling more rapid deployment of new techniques to computing systems, which would benefit end users.

Evaluation. We design Terracotta for a DDR5-based system [310,311] and evaluate it in two settings. First, we evaluate four techniques from four distinct domains: (i) PRADA [104,105] (PuD, §8.1), (ii) MoPAC-C [221] (low-cost DRAM maintenance, §8.2), (iii) MASA [48] (subarray-level parallelism, §8.3), and (iv) ChargeCache [279] (latency reduction, §8.4). Second, we evaluate a composition of two different DRAM techniques: MASA [48] and ChargeCache [279].

We compare the performance and energy efficiency of each Terracotta-based implementation against (i) a conventional DRAM system and (ii) a dedicated custom implementation, using cycle-level simulations with Ramulator 2.0 [316–323], integrated with DRAMPower [343,344]. We make three key observations from our comprehensive evaluation. First, Terracotta-based implementations of the four techniques and the composi-

tion of ChargeCache and MASA retain over 96% of their respective custom implementations’ performance benefits over the baseline. Second, the Terracotta-based composition of ChargeCache and MASA outperforms the individual Terracotta-based implementations of ChargeCache and MASA, showing that Terracotta’s flexibility can enable the benefits of multiple techniques without repeated modifications. Third, Terracotta’s flexibility incurs a *small* DRAM energy consumption overhead of 3.2% for PRADA and <1% for MoPAC-C, MASA, and ChargeCache relative to their respective custom implementations.

Terracotta’s flexibility comes at *low* hardware complexity. Terracotta’s custom command encodings require *no* additional pins, leveraging existing reserved command encoding space in the DDR5 standard (§6.1). We synthesize Terracotta’s programmable memory controller at the RTL level using the OpenROAD flow [345] and the Nangate 45nm library [346], and scale it to 10nm using DeepScale [347]. Our results show that, relative to a high-end server-grade CPU [348], Terracotta incurs *only* 0.03% die area and 0.56% thermal design power (TDP) overheads on top of a design that provides custom memory controller support for the four DRAM techniques we evaluate. Terracotta’s DRAM-side decoding logic adds *only* 5 flip-flops and fewer than 24 gates to the DRAM center stripe.

In this work, we make the following contributions:

- We propose **Terracotta**, a new memory control framework that reduces the repeated modifications needed to adopt diverse DRAM techniques by introducing flexibility into the DRAM interface and memory controller.
- We introduce **custom command extensions** that extend the DRAM interface through a one-time change, decoupling the structural format of new commands from their functional meaning and allowing vendors to define new commands without further interface changes.
- We identify four fundamental **memory controller primitives**: *state*, *triggers*, *updates*, and *actions*, that capture the common operational workflows of DRAM techniques. We expose programmatic control over them, allowing system designers to deploy new techniques post-silicon fabrication.
- We design Terracotta for a DDR5-based system and show that it retains almost all of the performance benefits of custom implementations, composes multiple techniques, and incurs low hardware complexity and DRAM energy overheads over custom implementations.

We hope and expect that Terracotta will inspire academia and industry to develop and adopt innovative DRAM techniques and optimizations that unlock improvements in system performance, energy efficiency, and robustness. By helping reduce stagnation in technique adoption caused by prohibitive standardization processes and rigid interfaces and memory controllers, Terracotta can accelerate progress from new ideas to practical benefits. To this end, we make Terracotta freely available at <https://github.com/CMU-SAFARI/Terracotta>.

2. Background

2.1. DRAM Organization and Operation

Fig. 1 shows a DRAM-based computing system.

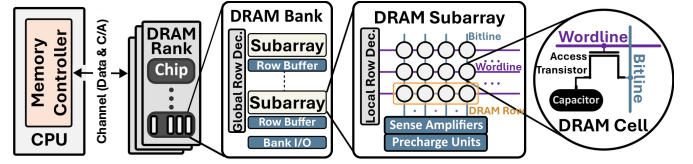


Figure 1: DRAM-based computing system.

DRAM Organization. DRAM-based systems are typically managed by multiple *Memory Controllers* (MCs). Each MC communicates with a *DRAM channel* through an I/O bus that consists of the Command/Address (C/A) bus, to issue commands and addresses, and the data bus, to transfer data. A DRAM channel consists of one or more *DRAM ranks*, each of which is a group of DRAM chips that operate in lockstep. A DRAM chip may be divided into multiple *DRAM bank groups* [309–311,315] (not shown in Fig. 1). A DRAM bank group is further divided into multiple *DRAM banks*, each of which is divided into multiple two-dimensional DRAM *subarrays* that share the bank’s I/O circuitry. Within each subarray, DRAM cells are organized into *rows* and *columns*. A DRAM row consists of DRAM cells that are connected to the same *wordline*. A DRAM cell connects to a *sense amplifier* via an *access transistor* and a *bitline*, and all sense amplifiers in a subarray form a *row buffer*. Within each cell, data is stored as *electrical charge* in the *capacitor*.

DRAM Operation. Reading or updating a DRAM cell requires *activating* (i.e., opening) a DRAM row. To open a row, the memory controller issues an activate (ACT) command with the bank group, bank, and row addresses. The *global row decoder* and *local row decoder* of the addressed bank select one of the many wordlines based on the row address. When the wordline is selected, the cell capacitor connects to the bitline and shares its charge, causing a perturbation in the bitline voltage. Then, the sense amplifier senses and amplifies the bitline voltage deviation towards V_{DD} or 0 V, fetching the row’s data into the row buffer and *restoring* the DRAM cell’s charge. The memory controller can then read or modify data in the row buffer using read (RD) and write (WR) commands. To close a row, the memory controller issues a precharge (PRE) command. To ensure correctness, the memory controller obeys the DRAM timing constraints [48,282,284,301,304,349–351] specified by the Joint Electron Device Engineering Council (JEDEC) [307–323], an industry standards body that includes DRAM vendors and system designers. We discuss four key timing constraints. First, the latency between an ACT command and subsequent RD and WR commands to the same bank must be at least the activation latency (tRCD). Second, the latency between consecutive RD commands (and consecutive WR commands) must be at least the column-to-column delay (tCCD). Third, the latency between a PRE command and a subsequent ACT command to the same bank must be at least the precharge latency (tRP). Fourth, the latency between consecutive ACT commands to the same bank must be at least the row cycle time (tRC).

2.2. Memory Controller Organization

The memory controller translates memory requests into DRAM commands and schedules them onto the DRAM channel. Fig. 2 shows the organization of a memory controller.

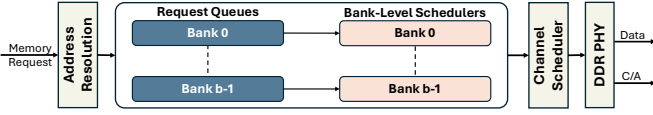


Figure 2: Memory controller organization.

The memory controller uses *address resolution* to convert each memory request’s physical address to its corresponding DRAM address, and places the request into a *request queue*. The request queue can be organized as multiple bank-partitioned queues or a single unified queue [301,349]. The memory controller then uses a *DRAM scheduler* to decide which command to issue to the DRAM channel. Typically, the scheduler has two levels [301,349]. The first level consists of *per-bank schedulers*, each of which selects the highest-priority request for its bank and determines the next valid command (e.g., ACT, RD, PRE) that the request needs. The second level is the across-bank *DRAM channel scheduler*, which selects the highest-priority per-bank valid command to issue. One popular prioritization algorithm employed by DRAM schedulers is FR-FCFS [300,301,349]. The *DDR Physical layer (PHY)* converts digital channel-level scheduled commands to analog I/O signals on the DRAM channel.

3. Motivation & Goal

Many prior works leverage the internal structure of DRAM to improve the performance, energy efficiency, and robustness of modern systems. We call such works *DRAM techniques*. These techniques aim to (i) enable in-memory computation by leveraging DRAM’s inherently highly parallel internal structure [8–12,34,35,69–114], (ii) improve DRAM availability and performance by reducing the overheads of maintenance operations such as refresh, read disturbance mitigation, memory scrubbing, and online error profiling [36–47,115–275], (iii) expose internal DRAM parallelism [48,230–232,276,277] to the host system, and (iv) reduce memory access latency by introducing or identifying fast-access regions [220,278–295].

Typically, these techniques rely on support from two parts of the system: (i) the DRAM interface [182] and (ii) the memory controller [4,300–306]. First, various DRAM techniques [34,48,104–110,217–221,232–235,276,278,290–299] expose novel DRAM functionalities to the memory controller by proposing new DRAM commands. These commands are *non-standard* because the existing DRAM interface *only* supports JEDEC-standardized DRAM commands [307–323]. For example, a recent prior work [104], PRADA, enables low-cost in-DRAM operations by proposing new DRAM commands. One such command is the A* command which executes only a subset of steps of the standard ACT command: it drives the word-line without performing sense amplification. Second, many techniques [34–39,44,45,48,69–110,182–258,270–289,292–299] require dedicated hardware support from the memory controller to determine *when* to invoke and *how* to perform them. For instance, ChargeCache [279], a memory latency reduction technique, introduces a metadata table to track recently precharged rows and applies lower timing constraints when accessing such recently precharged rows.

The support that DRAM techniques require from the DRAM

interface and controller poses two deployment obstacles. First, a DRAM technique that needs a new DRAM command requires the development of a new DRAM standard or a revision of an existing standard [307–323]. Such a change can be prohibitive because it requires agreement among *all* the stakeholders of the DRAM standard (e.g., DRAM vendors and system designers). These stakeholders may have conflicting goals, approaches, and interests [5,6,181,182], potentially hindering the adoption of a technique that benefits a subset of the stakeholders. Second, even if the stakeholders of the standard agree on a technique, already-deployed computing systems *cannot* easily adopt the technique, since such systems remain in the field for several years [216,324–326]. This problem is particularly acute for remote deployments such as satellite [327,328] and space systems [329–339] that *cannot* be easily upgraded and may remain in the field for decades. Over these long deployment lifetimes, already-deployed systems may benefit from new DRAM techniques to meet the fast-evolving memory demands of various workloads (e.g., AI and ML) [18,29,33,52,58–61,73,340,341]. Yet these systems cannot adopt such techniques.

The DRAM interface and memory controller of current computing systems are *rigid*: they support *only* standard DRAM commands and control logic that is defined at design time. This design philosophy hinders the deployment of DRAM techniques in two ways. First, already-deployed systems *cannot* adopt a new DRAM technique, even after the technique is standardized. Second, future systems can adopt a technique *only* by modifying the DRAM interface and redesigning the memory controller. Adopting N DRAM techniques one after another therefore requires up to N interface changes and N controller redesigns. These *repeated* modifications deprive computing systems of performance improvements, energy efficiency gains, and essential security features [6,43,182,215,216,342], motivating the need for a *less rigid* memory system.

Our goal is to reduce the repeated DRAM interface and memory controller modifications that hinder or slow down the deployment of diverse DRAM techniques. To this end, the memory system needs to be able to (i) support new DRAM commands *without* a DRAM interface modification for each new command, and (ii) implement memory controller support for new DRAM techniques even *after* the controller is fabricated.

4. Terracotta

Our analysis of 32 DRAM techniques [34–36,48,80–83,104,220–231,278–282,294–298] across seven different categories reveals that these techniques share similar structures in how they define new commands and how they use the memory controller. Our **key idea** is to leverage these similarities to compose a set of configurable *primitives* that DRAM vendors and system designers can use to implement many diverse DRAM techniques. To this end, we propose **Terracotta**, a new memory control framework that consists of two flexible components: (i) custom command extensions that enable DRAM vendors to define new commands within a single, standardized interface extension (§4.1), and (ii) a programmable memory controller that enables system designers to express controller support for new techniques post-silicon fabrication (§4.2).

4.1. Custom Command Extensions

We observe that DRAM commands from various techniques can be decomposed into three primitives: (i) *operation type*, (ii) *target DRAM locality*, and (iii) *payload*. We illustrate these using PRADA’s [104,105] non-standard activation command, A*. The operation type specifies the operation DRAM must perform. For A*, the operation drives a wordline without sense amplification. The target DRAM locality identifies the portion of DRAM targeted by the operation. For A*, the target DRAM locality is the target bank. The payload specifies the operation’s operands, such as row, column, or subarray addresses within the target DRAM locality. For A*, the payload is the row address corresponding to the target wordline.

The existing DRAM interface standardizes both the *command structure*, which specifies the bit positions and widths of the fields encoding these three primitives, and their *functional meaning*, which specifies how DRAM interprets the values in those fields. Thus, adding a new command to the DRAM interface requires standardizing both its structure and meaning. To avoid this, Terracotta augments the existing interface through a *one-time* change that standardizes only the structure of custom commands, leaving DRAM vendors to independently define each primitive’s functional meaning. For example, a vendor can define PRADA’s A* by assigning an operation type value to “activation without sense amplification” and interpreting the locality and payload fields as the target bank and row address, respectively. This separation allows DRAM vendors to independently define custom commands within a common command structure *without* further interface standardization.

4.2. Programmable Memory Controller

Many DRAM techniques share common memory controller structures and operational workflows despite proposing dedicated hardware. We formulate these commonalities into four fundamental primitives: *state*, which provides microarchitectural information required by techniques, and *triggers*, *updates*, and *actions*, which form a workflow that executes on every scheduled command. Fig. 3 shows these primitives, their responsibilities, and their interactions.

State is the microarchitectural information the memory controller maintains to execute DRAM techniques. It consists of: (i) *metadata*, which tracks information about DRAM operations

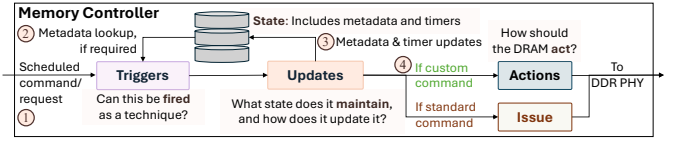


Figure 3: Terracotta’s memory controller primitives, their responsibilities, and their interactions.

(e.g., access history, counters), and (ii) *timers*, which ensure that DRAM commands meet timing constraints. For example, ChargeCache [279] maintains metadata to track recently precharged rows, and PRADA [104,105] requires timers to enforce the timing constraints of its non-standard commands.

When a command or request is scheduled ①, *triggers* evaluate *conditions* to determine whether a technique should fire. A condition typically involves: (i) the *scheduled command or request* type, and (ii) a *metadata* lookup ② from state. For example, ChargeCache [279] fires when a read (RD) request targets a row present in its metadata table of recently precharged rows. Irrespective of whether a trigger fires, *updates* perform the technique’s *state maintenance* operations. This includes: (i) maintaining metadata consulted by triggers, and (ii) updating timers to ensure that subsequent commands satisfy timing constraints ③. For example, ChargeCache performs two updates: (i) inserting precharged rows into a metadata table, and (ii) recording timing constraints for future accesses to such rows. If a technique requires a non-standard DRAM operation ④, the controller provides support for custom commands via *actions*. For example, PRADA’s action issues a custom A* command to drive a wordline without sense amplification.

Since these primitives capture common controller structures and are *not* tied to specific functionalities, Terracotta can expose them as configurable building blocks rather than designing separate fixed logic blocks for each technique. Doing so allows system designers to *program* state, trigger conditions, update rules, and actions to implement new DRAM techniques, reducing the need to modify the memory controller. We show next that Terracotta’s primitives cover a wide range of DRAM techniques.

4.3. Applicability Across Techniques

Table 1 maps the 32 DRAM techniques we analyze to Terracotta. Each row breaks down a category by its required state, triggers,

Table 1: 32 DRAM techniques across seven categories and how they map to Terracotta.

Category	State	Triggers	Updates	Actions
Processing-using-DRAM [34,35,81,82,104]	Timers for custom commands	Fire on a PuD request (e.g., COPY)	Reload timers when a custom command is issued	Multiple row activation and compute commands
Probabilistic controller-side read disturbance mitigation [36,221]	Single-bit register to mark a preventive action	Fire when register is set	Probabilistic setting of a register	Lower-overhead preventive actions
Subarray-level parallelism [48,230,231]	Set of parallelizable requests; Timers for custom commands	Fire on parallelizable requests	Track parallelizable requests; Reload timers	Subarray-level control commands
Access latency reduction [220,278–282,294,295]	Fast-access row table; May require timers	Fire on fast-access row table hits	Insert/evict fast-access rows; Use lower timing constraints	Fast-access custom commands
Deterministic controller-side read disturbance mitigation [36,222–229]	Activation counters	Fire when a counter exceeds a threshold	Increment activation counters and reset on refresh	Lower-overhead preventive actions
In-DRAM true random number generation (RNG) [80,83]	Timers for custom commands	Fire on an RNG request	Reload timers when a custom command is issued	—
Fine-grained activation [296–298]	—	Fire on activations	—	Fine-grained activation commands

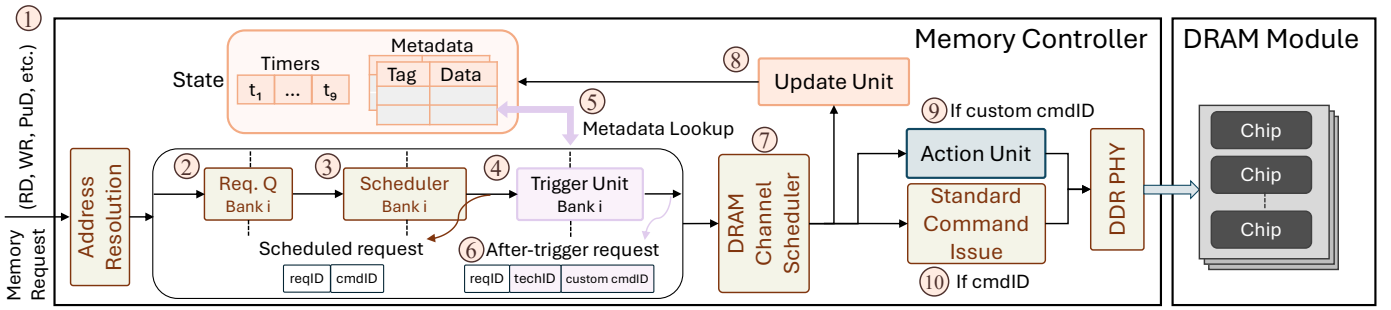


Figure 4: Terracotta: end-to-end memory request flow.

updates, and actions (and custom commands).¹ While these techniques serve diverse goals, they rely on common underlying structures that Terracotta captures through its command extensions (§4.1) and controller primitives (§4.2).²

5. Terracotta: Design Overview

Fig. 4 illustrates the end-to-end flow of memory requests at runtime through Terracotta’s memory controller. The memory controller receives memory requests ① and, similar to conventional memory controllers (§2), places them into per-bank request queues ②. The per-bank scheduler ③ selects a request and passes it to the corresponding trigger unit ④. The trigger unit evaluates the request against a set of conditions using request attributes such as request ID (*reqID*), command ID (*cmdID*), and DRAM addresses. For techniques that depend on metadata, the trigger unit performs a metadata lookup ⑤. If the request satisfies the trigger conditions, the trigger unit marks it as eligible for the corresponding technique ⑥ in two steps. First, it adds a technique ID (*techID*) to identify the triggered technique. Second, if the technique requires a custom command, the trigger unit replaces the request’s *cmdID* with the corresponding custom command identifier, *custom cmdID*. The trigger unit then forwards both marked and unmarked requests to the channel scheduler ⑦. The channel scheduler selects a request and simultaneously forwards it to the update unit and the action unit. The update unit manages memory controller state ⑧, such as metadata and timers, based on the scheduled command’s *techID* and *cmdID*. If the selected command is a custom command ⑨, the action unit uses its *cmdID* to issue the corresponding custom command to the DDR PHY according to Terracotta’s custom command extension format. Otherwise, if the selected command is a standard command ⑩, Terracotta issues it to the DDR PHY through the standard command issue path.

6. Terracotta: Implementation

A key challenge in designing Terracotta is balancing the flexibility needed to support diverse DRAM techniques with the hardware complexity required to provide that flexibility. This

trade-off raises three key implementation questions, each of which exposes a corresponding design space.

How should custom commands be encoded? Several DRAM standards reserve command encoding space for future use [307–315]. A design that uses this encoding space can support custom commands without adding pins to the C/A bus, but is limited by the size of the reserved space. Alternatively, a design can add pins to the C/A bus to provide a larger encoding space, at the cost of additional area.

How should controller state be managed across techniques and banks? One design choice is to share the controller state across techniques and banks. However, this may lead to contention for the shared state, and leave performance on the table in the absence of an intelligent prioritization and fairness scheme. Alternatively, a design can partition the state across techniques and banks, avoiding contention but potentially leaving some state unused.

How expressive should the trigger, update, and action logic be? The design space spans increasing flexibility, from programmable tables to Coarse-Grained Reconfigurable Arrays [352–356], embedded Field Programmable Gate Arrays [357,358], and in-order cores [359]. With greater flexibility, the implementation of more complex, future techniques may be easier but at the cost of increased hardware complexity.

Next, we present a concrete implementation of Terracotta for a system based on DDR5 [310,311], a representative modern DRAM specification. Our implementation uses DDR5’s reserved command encoding space and programmable tables to maintain low hardware complexity, while partitioning state across techniques and banks to maximize their benefits. We describe the key components of Terracotta’s implementation in detail, including: (i) the custom command encoding (§6.1), (ii) the programmable controller microarchitecture (§6.2), and (iii) boot-time system support for configuring DRAM techniques (§6.3). We conclude the detailed design with a discussion on how Terracotta’s programmable memory controller does *not* expand the system’s attack surface (§6.4) and of Terracotta’s hardware complexity (§6.5).

6.1. Custom Command Encoding

Fig. 5 shows the command encodings Terracotta uses for custom DRAM commands. The DDR5 C/A bus reserves command encoding space [310,311], identified by the 01 pattern in the two most significant bits ①. Terracotta leverages this reserved space for its custom command encoding. The encoding com-

¹Custom commands are tied to actions because an action provides the controller-side logic to issue the custom command.

²Some DRAM techniques propose to also modify the DRAM chip. Of the 32 techniques in Table 1, 15 require DRAM chip modifications. Terracotta provides the DRAM interface and memory controller support required by all 32 techniques, and deliberately leaves the DRAM chip modifications to the DRAM vendors. We discuss this further in §9.2.

prises (i) a *header* and (ii) a *payload*, each occupying one C/A bus cycle. Each custom command consists of a header followed by one or more payloads, one for each operation operand.

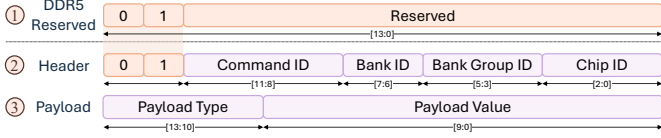


Figure 5: Custom-command structural encoding.

Header ②. The header uses the reserved 01 pattern to signal a custom command to the DRAM. The header serves three purposes. First, it conveys the operation type using a 4-bit command ID, allowing a DRAM vendor to define up to 16 different custom commands for each DRAM chip design.³ Second, because each operation type is associated with a fixed number of payloads, the command ID field of the header *implicitly* determines the number of payloads that follow. Third, the header encodes the target DRAM locality using bank ID and bank group ID fields.⁴

Payload ③. A custom command may require multiple *different* operands, such as row and subarray addresses. Thus, each payload associates a payload value with a 4-bit payload type identifier, allowing a DRAM vendor to define up to 16 different payload types per custom command.

6.2. Programmable Controller Microarchitecture

In this subsection, we describe the microarchitectural implementation of Terracotta’s memory controller primitives.⁵

6.2.1. State: Metadata and Timers. Our implementation partitions Terracotta’s memory controller state across techniques and DRAM banks to mitigate resource contention, avoiding the need for a fairness scheme (§6). The per-bank state comprises two hardware components: (i) *metadata tables* and (ii) *timers*. First, the metadata table is a 64-entry, 2-way set-associative structure. Each entry consists of three fields: tag, data, and replacement bits. To prevent stale data, we employ a configurable age-based entry invalidation policy. Second, the timers consist of 16 programmable down-counters that enforce the timing constraints introduced by custom commands. We note that the metadata table capacity (64 entries) and the number of down-counters (16) are *design-time parameters*, analogous to sizing a processor cache. Scaling these structures to accommodate techniques with larger state requirements does *not* require changes to Terracotta’s programmable logic (triggers, updates, and actions) or custom command encoding.

6.2.2. Trigger Unit. A trigger serves three functions. First, *request evaluation* checks each request selected by a bank-level scheduler against the trigger conditions of the configured techniques to determine the set of eligible techniques. Second, *C/A*

bus filtering excludes some eligible techniques to mitigate critical request starvation, since custom commands may occupy multiple C/A bus cycles (§6.1). Third, *technique prioritization* selects one technique according to a programmable priority order. Fig. 6 illustrates the trigger unit microarchitecture.

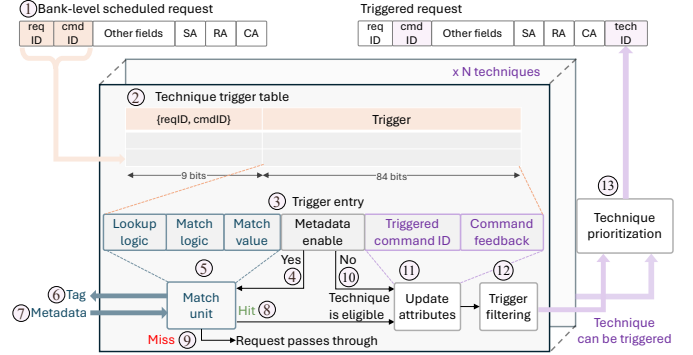


Figure 6: Microarchitecture of the trigger unit.

The trigger unit is partitioned across techniques to evaluate multiple techniques against their trigger conditions in parallel. **Request evaluation.** The *reqID* and *cmdID* fields of the bank-level scheduled request ① form the tag used to search each technique’s *technique trigger table* ②. The trigger table is a programmable, fully associative structure containing 16 *trigger entries*. The table returns a trigger entry ③ on a hit. The entry consists of six fields: (i) metadata enable flag, (ii) lookup logic, (iii) match logic, (iv) match value, (v) triggered command ID, and (vi) command feedback. For metadata-dependent techniques, the *metadata enable* flag of the trigger entry is set ④. The trigger unit evaluates a metadata match using the *match unit* ⑤ in two steps. First, it generates a lookup tag by concatenating up to three DRAM address fields of the request, i.e., subarray address (SA), row address (RA), and column address (CA), as configured by the *lookup logic* field, and uses this tag to look up the technique’s bank-partitioned metadata table ⑥ (§6.2.1). Second, on a metadata table hit, the match unit evaluates the retrieved metadata ⑦ against the *match value* using either a direct comparison or a constant modulo operation, as determined by the *match logic* field. On a metadata match ⑧, the technique becomes *eligible*. Otherwise, the request proceeds with its original standard command ⑨. For metadata-independent techniques ⑩, a technique trigger table hit alone makes the technique eligible. For each eligible technique, the trigger unit updates the request’s attributes ⑪ in two ways. First, if the technique uses a custom command, the trigger unit replaces the request’s *cmdID* with the *triggered command ID* specified in the trigger entry. Second, the trigger unit provides *command feedback* to the scheduler, which is either the command that must follow the custom command (e.g., the read after a custom activate) or an indication that the custom command has completed the request.

C/A bus filtering. Some techniques require custom commands that occupy multiple C/A bus cycles (§6.1). Such techniques can hog the C/A bus, potentially starving critical requests that do not use custom commands, degrading system performance. To mitigate this issue, the trigger unit evaluates

³DDR5 [310,311] also provides 4 reserved-for-future-use (RFU) commands that can further expand the encoding space for custom commands.

⁴The header also carries the Chip ID (CID) field to support 3D-stacked DRAM configurations, following the standard DDR5 command structure.

⁵We assume component sizes sufficient to support up to four techniques concurrently. Supporting more techniques requires scaling these components and does *not* require changes to the command encodings or controller logic.

whether an eligible technique should be filtered out based on C/A bus utilization (12). The trigger unit employs a channel-level sliding-window counter that tracks the fraction of C/A bus cycles occupied by commands over a configurable window. When this fraction exceeds a programmable per-technique threshold, the technique is filtered out and the request proceeds with its original standard command.⁶

Technique prioritization. If multiple techniques remain eligible for a request, the trigger unit selects one based on a programmable priority order (13), and marks the request with the selected technique’s *techID*. This is useful in prioritizing security-critical techniques.

Example: For ChargeCache [279], the trigger is metadata-dependent. On a scheduled ACT command, the lookup logic concatenates the subarray address (SA) and row address (RA) to look up the metadata table, which stores recently precharged rows. A hit indicates the row is still highly charged. The match logic compares the entry against the match value to confirm its validity. If the entry is valid, the trigger unit marks the request with the *techID* to indicate a ChargeCache hit and provides command feedback to the scheduler.

6.2.3. Update Unit. An update maintains the memory controller state based on channel-level scheduled commands. It serves two functions. First, *metadata update* modifies the data stored in the technique’s metadata table. Second, *timer update* loads timers with its technique-specific timing constraints. Fig. 7 illustrates the update unit microarchitecture.

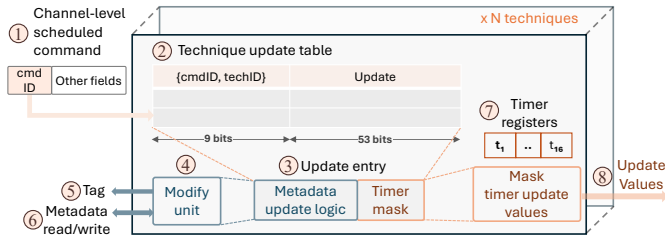


Figure 7: Microarchitecture of the update unit.

The update unit is partitioned across techniques so that all per-technique updates are performed in parallel. For every channel-level scheduled command (1), the update unit uses the *cmdID* and *techID* fields to form the tag used to search each technique’s *technique update table* (2). Each technique update table is a programmable, fully associative structure containing 8 *update entries*. On a hit, the update unit performs technique-specific state updates based on the update entry’s two fields (3): (i) metadata update logic, and (ii) timer mask.

Metadata update. For techniques that maintain metadata, the update unit uses the *metadata update logic* field to configure an arithmetic unit called the *modify unit* (4). The modify unit performs a read-modify-write operation in two steps. First, it generates a lookup tag like the trigger lookup logic (§6.2.2) and uses this tag to look up the technique’s bank-partitioned metadata table (5). Second, it modifies the retrieved data using one of three operations: adding a constant, setting it to a

specified value, or setting it to a specified value with a specified probability. The modify unit then writes the result back to the metadata table (6).

Timer update. The update unit employs dedicated per-technique *timer registers* (7) to store timer update values for techniques that introduce new timing constraints or modify existing ones. The *timer mask* field of the update entry selects which of the bank’s timers (§6.2.1) are loaded with these values.

Example: ChargeCache [279] performs two updates. First, on a PRE command, the update unit inserts the currently open row into the metadata table to track highly charged rows. Second, on a ChargeCache ACT command, the updates unit applies reduced timing constraints that are stored in the timer registers.

6.2.4. Action Unit. An action enables the memory controller to issue a custom command over the C/A bus. Each custom command consists of two parts: *header* and *payloads* (§6.1). Fig. 8 illustrates the action unit microarchitecture.

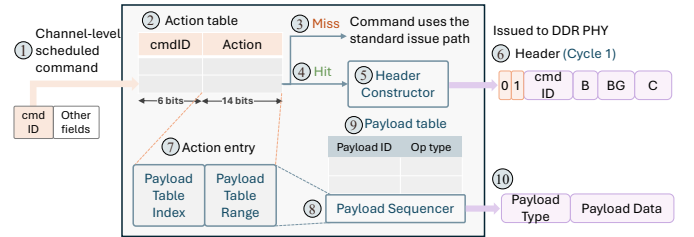


Figure 8: Microarchitecture of the action unit.

For every channel-level scheduled command (1), the action unit uses the *cmdID* field as the tag to search the *action table* (2), a programmable, fully associative structure containing 16 *action entries*. On an action table miss (3), the command proceeds through the standard issue path. On a hit (4), the action unit performs two steps. First, the action unit uses the header constructor (5) to populate the header based on the custom command encoding (6) and dispatches it to the DDR PHY. Second, the action unit constructs payloads and sequences them over the C/A bus using the two attributes of the action entry (7): (i) *payload table index* and (ii) *payload table range*. These attributes direct the payload sequencer (8) to a contiguous set of entries in the *payload table* (9), a programmable structure with 128 entries that maps fields of the scheduled command (e.g., row, column, subarray) to the payload identifiers defined by the DRAM vendor (§6.1). For each selected entry, the payload sequencer extracts the operand from the scheduled command, pairs it with its payload identifier, and dispatches the resulting payload to the DDR PHY in the cycles following the header (10).

Example: PRADA [104,105] uses actions to issue the custom command A*. On an A* command, the action table returns an action entry. The action unit constructs the header to specify the *cmdID* and the DRAM locality targeted, and subsequently sequences the payloads that identify the subarray and row targeted by the A* command.

6.3. System Support

To support post-silicon flexibility, Terracotta leverages system firmware to program the memory controller during the Pre-EFI

⁶The filter can be disabled per technique, ensuring that security-critical and execution-critical (e.g., PuD) techniques always fire when triggered.

Initialization (PEI) phase [360]. This industry-standard early boot stage occurs before the system memory is initialized, allowing the processor to execute code directly from the motherboard’s Serial Peripheral Interface flash (SPI flash) [361]. Terracotta programs the memory controller in three steps. First, the firmware queries the Serial Presence Detect (SPD) [307–315] of each installed Dual In-line Memory Module (DIMM) to identify its vendor and module serial number. Second, the firmware matches the module against a repository of *configurations* signed by the system designer and stored in the SPI flash. Each configuration contains the specific trigger, update, action, and payload table entries, along with timer update values for that DIMM. Third, the firmware programs the controller’s tables with the identified configuration via Memory-Mapped I/O [362], which provides a standard interface for the CPU to configure the controller’s internal logic before the DRAM is online. In cases where a module’s SPD information is unrecognized or a configuration is missing, the controller safely defaults to a transparent JEDEC-compliant mode, which preserves system stability and backward compatibility with standard memory. As new DIMM modules and security vulnerabilities emerge, a firmware update can deploy new logic without hardware changes. This follows the established model for deploying microcode updates in modern processors [362].

6.4. Security

Terracotta’s programmable memory controller does *not* expand the system’s attack surface beyond that of a dedicated custom memory controller. Our threat model assumes that the system designer and the DRAM vendor are *trusted* entities. The attacker attempts to alter the memory controller’s programmable tables, for example, to disable a security technique, but *cannot* decrypt secure communication. Such an attacker *cannot* alter the tables for four reasons. First, the DIMM configurations (§6.3) are authored and signed by the system designer, who already provides the platform firmware and is trusted. Second, the platform’s Hardware Root of Trust [363] maintains the integrity of the DIMM configurations, by ensuring that only cryptographically signed configurations are programmed into the memory controller. Third, *only* the Pre-EFI Initialization (PEI) system software can program Terracotta’s programmable tables (§6.3). After the PEI phase completes, register lock bits [364] lock the Memory-Mapped I/O registers backing the tables until the next platform reset, so no malicious operating system or user application can modify them. Fourth, a custom command provides no more control over the DRAM than the dedicated command it replaces, because the DRAM vendor still defines what each command does (§6.1).

6.5. Hardware Complexity

DRAM Interface. Terracotta leverages DDR5’s existing reserved command encoding space (§6.1) and introduces *no* additional physical pins to the DRAM C/A bus.

Memory Controller. To estimate area, power, and latency overheads, we implement Terracotta in RTL with support for up to four techniques. We synthesize the design to GDSII using the OpenROAD flow [345] and the Nangate 45nm library [346],

and scale the results to a 10nm node using DeepScale [347]. The synthesized design occupies 0.083mm² and consumes 324mW of static power per DDR5 channel, which is *only* ~0.04% of the chip area and ~0.67% of the power budget of a 10nm Intel Xeon processor [348]. Of the three primitives, triggers add two cycles to the critical path of a memory controller that does not support any techniques. Updates remain off the critical path for techniques whose minimum timing constraint exceeds the two-cycle metadata read-modify-write latency. Actions are off the critical path because the custom and standard command issue paths operate in parallel.

DRAM Logic. Terracotta confines the DRAM-side custom command decoding logic to the DRAM chip center stripe [365]. Although individual DRAM techniques may require modifications to the bank peripherals or cell arrays, the custom command extensions require changes only to the center stripe. The custom command decoder comprises a finite-state machine with one bit of state, a 4-bit payload counter to track up to 16 payloads per custom command (§6.1), and fewer than 24 gates for combinational control logic. For context, a single 8-bit DDR5 mode register requires eight flip-flops, and the standard [310,311] defines 256 such registers. By restricting the custom command decoder to the center stripe, Terracotta does *not* significantly increase DRAM manufacturing complexity beyond what the DRAM techniques themselves require.

7. Evaluation Methodology

We evaluate whether Terracotta retains the benefits that various DRAM techniques provide with customized implementations. To do so, we select one representative technique each from four domains of DRAM techniques: (i) processing-using-DRAM (PRADA [104,105], §8.1), (ii) lower-overhead DRAM maintenance operations (MoPAC-C [221], §8.2), (iii) subarray-level parallelism (MASA [48], §8.3), and (iv) access latency reduction (ChargeCache [279], §8.4).

Simulator. We conduct cycle-level performance and energy simulations using Ramulator 2.0 [316–323], integrated with DRAMPower [343,344]. Table 2 presents the baseline system configuration used in our evaluation, unless stated otherwise.

Table 2: Baseline System Configuration

Processor	4.2 GHz, 1- or 4-core, 4-wide issue, 256-entry instruction window
LLC	64B cache-line, 16-way set-associative, 2 MiB/core
Memory Controller	64-entry read/write request queues Scheduling policy: FR-FCFS + column cap of 4 [300,301,349] Address mapping: RoBaRaCoCh
DRAM	DDR5-3200 [310,311], 1 channel, 1 rank, 8 bank groups, 4 banks/bank group, 128 subarrays/bank, 64K rows/bank

We further extend Ramulator 2.0 to model: (i) Terracotta-based implementations of the four techniques described in §8.1–§8.4, (ii) custom hardware implementations for these techniques, and (iii) *ideal* implementations of these techniques, wherever applicable. To ensure a fair comparison, we rigorously model the latency overheads of each configuration. For custom implementations of DRAM techniques, we incorporate latency overheads from their respective memory controller RTL models and optimistically assume 2 cycles for non-standard DRAM commands introduced by these techniques. For the Terracotta-based implementations, we incorporate the

two cycle memory controller latency overhead derived from our RTL synthesis (§6.5) and determine their command latencies based on the number of payloads they require (§6.1).

Workloads. We evaluate 57 single-core workloads from five benchmark suites: SPEC CPU2006 [366], SPEC CPU2017 [367], TPC [368], MediaBench [369], and YCSB [370]. We group these single-core workloads into two memory-intensity groups based on their row buffer misses per kilo instructions (RBMPKI): *Low* (RBMPKI < 2) and *High* (RBMPKI ≥ 2). We combine randomly selected single-core workloads to construct 60 multi-programmed four-core mixes belonging to five categories, HHHH, HHHL, HHLL, HLLL, and LLLL, with 12 mixes per category. For PRADA, we curate five microbenchmarks (discussed in more detail in §8.1).

Metrics. We use Instructions Per Cycle (IPC) and weighted speedup [371–373] to evaluate single- and multi-core system performance, respectively.

8. Evaluation

In this section, we: (i) discuss the four DRAM techniques we evaluate and compare the performance of their Terracotta-based implementations to their custom implementations (§8.1–§8.4); (ii) show how Terracotta can compose the performance benefits of multiple techniques by combining ChargeCache and MASA (§8.5); (iii) analyze Terracotta’s energy (§8.6.1), latency (§8.6.2), and area and power (§8.6.3) overheads relative to custom implementations; (iv) evaluate Terracotta’s performance sensitivity to memory controller latency overhead (§8.6.4); (v) analyze an alternative Terracotta implementation that uses Coarse-Grained Reconfigurable Arrays (CGRAs) [352,353] to realize trigger and update units, trading area and latency for more expressive matching and update logic (§8.7).

8.1. Processing-using-DRAM: PRADA

Processing-using-DRAM (PuD) is an emerging paradigm that alleviates the data movement bottleneck [18,22–24,28–30,33,374] between processing elements and main memory by performing massively parallel in-DRAM operations [8–12,34,35,69–114], including data copy [34,113,375] and bitwise operations [35,69,110,112]. These bitwise operations are performed by simultaneously activating multiple rows in a subarray. Ambit [35,110] enables in-DRAM bulk bitwise computation but requires two costly DRAM modifications: (i) split row decoders to activate multiple rows simultaneously within a subarray and (ii) dual-contact cells to perform NOT operations.

PRADA. A recent work, PRADA [104,105], eliminates the need for both circuit modifications required by Ambit through two mechanisms. First, Sequential Row Activation (SRA) sequentially activates wordlines to share charge before sensing, obviating the need for a split row decoder. Second, PRADA introduces two new bitline sense amplifier states that enable NOT operations without dual-contact cells. PRADA introduces four DRAM commands to support SRA and NOT operations, including A* for activation without sensing and N for NOT.

Terracotta-based PRADA. Terracotta implements PRADA’s four new DRAM commands using its custom command exten-

sions. Each custom command conveys two payloads: (i) a subarray address and (ii) a row address. We configure Terracotta’s controller primitives as follows. The trigger unit identifies PuD requests, such as COPY, NOT, AND/OR, NAND/NOR, and XOR/XNOR. For each request, it sets the *cmdID* to the corresponding custom command ID and provides feedback to the scheduler to indicate which command should follow. The update unit maintains timers that enforce the timing constraints of SRA and NOT operations. The action unit issues the custom command sequences required for each PuD request.

Results. We compare the performance of the Terracotta-based and custom implementations of PRADA for in-DRAM computation. To evaluate how bank-level parallelism affects their performance, we run operations in one DRAM bank and across 32 DRAM banks. We evaluate five in-DRAM computation operations: (i) COPY, (ii) NOT, (iii) AND/OR, (iv) NAND/NOR, and (v) XOR/XNOR. For each operation, we run a microbenchmark⁷ that repeatedly performs the operation on large input vectors. We hold the input size per bank constant at 1 MB, resulting in total input sizes of 1 MB and 32 MB for the one DRAM bank and 32 DRAM bank configurations, respectively. For the CPU baseline, we run equivalent microbenchmarks that perform the same operations on the same input sizes. Fig. 9 shows the performance of the Terracotta-based PRADA implementation (labeled Terracotta-PRADA) and a custom PRADA implementation (labeled PRADA) for in-DRAM computation operations in 1 bank (left) and across 32 banks (right) normalized to the CPU baseline.

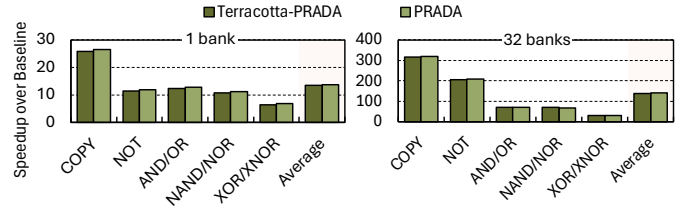


Figure 9: Performance of Terracotta-PRADA and custom PRADA for in-DRAM computation operations in 1 bank (left) and across 32 banks (right).

We draw two observations. First, for in-DRAM computation operations in 1 bank, Terracotta-PRADA closely matches PRADA’s performance. Terracotta-PRADA provides an average speedup of $13.36\times$ over the CPU baseline, compared to $13.79\times$ for PRADA, retaining 96.9% of PRADA’s average speedup. We attribute this gap to: (i) the memory controller latency overhead and (ii) three-cycle custom command latency (one header and two payloads) compared to custom PRADA’s two cycles (§7). Second, for in-DRAM computation operations across 32 banks, Terracotta-PRADA provides *nearly identical* performance to PRADA.⁸ The smaller performance gap indicates that scheduling independent work across banks largely

⁷We design a single microbenchmark for the AND and OR operations because they require the same command sequence. This also applies to NAND/NOR and XOR/XNOR [104,105].

⁸For COPY, NOT, and AND/OR microbenchmarks, Terracotta-PRADA outperforms PRADA by 0.2%–1.7%. We attribute these marginal gains to more favorable memory request scheduling resulting from differences in memory controller and command latencies between Terracotta-PRADA and PRADA.

hides the controller and command latency overheads exposed in the 1 bank case. We conclude that Terracotta’s latency overheads have a small performance impact on PRADA and that high bank-level parallelism can hide these overheads.

8.2. Lower-Overhead DRAM Maintenance Operations: MoPAC-C

RowHammer [7,13,16,36,40,115,116,376] occurs when a DRAM row is repeatedly activated more times than a pre-defined RowHammer threshold (T_{RH}) within a refresh interval, leading to bitflips in physically nearby victim rows [36]. A simple way to prevent bitflips is to preventively refresh potential victim rows using refresh commands such as REF or specific preventive refresh commands such as RFM [310]. To mitigate RowHammer, the April 2024 DDR5 specification [311] introduced Per-Row Activation Counting (PRAC) [39,185], a mitigation framework that maintains an activation counter for each DRAM row [36,115,116,136,218] to accurately determine when to *preventively refresh* victim rows. PRAC increments each row’s activation counter during precharge. When an activation counter reaches T_{RH} , PRAC triggers preventive refreshes using RFM commands [310] similar to approaches proposed in prior works [136,137,182,192,224]. However, updating activation counters during precharge increases the row cycle time (t_{RC}) and precharge latency (t_{RP}), leading to significant performance degradation [39,185].

MoPAC-C. To alleviate this performance degradation, MoPAC [221] proposes MoPAC-C, a memory controller-side technique that skips PRAC counter updates with a probability p . MoPAC-C preserves security guarantees by carefully selecting p based on the system’s RowHammer threshold (T_{RH}). Implementing MoPAC-C requires a non-standard DRAM command, precharge with counter update (PREcu), which the memory controller issues only when MoPAC-C selects a counter update.

Terracotta-based MoPAC-C. Terracotta implements MoPAC-C’s PREcu command using its custom command extensions. The custom PREcu command requires a single payload to convey the subarray address. Terracotta tracks pending updates in the metadata table. We configure Terracotta’s controller primitives as follows. The update unit sets a metadata table entry with probability p whenever an ACT is scheduled. The trigger unit monitors incoming requests and sets the *cmdID* of PRE commands to PREcu if the metadata table entry is set. The action unit issues the custom PREcu command.

Results. Fig. 10 shows the performance overheads of the Terracotta-based MoPAC-C implementation, a custom MoPAC-C implementation, and PRAC [39,185,377] for single-core workloads in subplot (a) and four-core workload mixes in subplot (b). The x-axis shows the workload types with varying memory intensities and the y-axis shows mean performance normalized to a baseline with *no* RowHammer mitigation. We evaluate all three implementations at three RowHammer thresholds (T_{RH}): 250, 500, and 1000. In each workload type, the patterned bars show the mean performance of Terracotta-based MoPAC-C (labeled Terracotta- $\{T_{RH}\}$), solid bars show the mean performance of custom MoPAC-C (labeled MoPAC- $\{T_{RH}\}$), the light gray bar shows the mean performance of PRAC, and error bars show

the standard error of the mean across all the workloads of the workload type. We show a single bar for PRAC because its performance does *not* vary across the evaluated thresholds [39]. *GMean* shows the geometric mean of normalized performance for each implementation across the workload types.

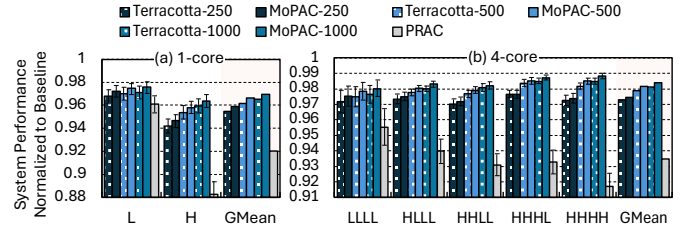


Figure 10: Performance impact of Terracotta MoPAC-C, custom MoPAC-C, and PRAC for three RowHammer thresholds: 250, 500, and 1000, on single-core and four-core workload mixes.

We draw three key observations. First, for single-core workloads and four-core workload mixes, Terracotta-based MoPAC-C closely matches the performance of MoPAC-C across all three RowHammer thresholds. Terracotta-based MoPAC-C provides geometric mean performance within 0.3% (0.5%) of custom MoPAC-C for four-core workload mixes (single-core workloads). Second, Terracotta-based MoPAC-C recovers a significant fraction of performance lost to PRAC across all T_{RH} values. At the lowest evaluated RowHammer threshold, 250, Terracotta-based MoPAC-C provides normalized geometric mean performance of 0.97 (0.95) compared to PRAC’s 0.93 (0.92) for four-core workload mixes (single-core workloads). This corresponds to a 4.1% (3.7%) performance improvement over PRAC for four-core workload mixes (single-core workloads). Third, Terracotta-based MoPAC-C retains a large fraction of the performance benefits of custom MoPAC-C over PRAC, retaining 96.0% (88.2%) of the custom implementation’s improvement for four-core workload mixes (single-core workloads) at a RowHammer threshold of 250. We conclude that Terracotta can effectively enable the benefits of techniques that perform new DRAM maintenance operations.

8.3. DRAM Parallelism: MASA

To serve multiple memory requests in parallel, modern DRAM systems employ multiple banks that can be accessed independently. However, memory requests to different rows in the same bank incur *bank conflicts* [48,378], requiring the memory controller to close the open row before activating another. Such bank conflicts can lead to a serialization of memory requests and row buffer thrashing, degrading performance [48]. While increasing the number of DRAM banks can reduce bank conflicts, this comes at a high cost [48].

MASA. To mitigate the performance impact of bank conflicts, MASA (Multitude of Activated SubArrays) [48] exposes DRAM subarrays to the memory controller and allows multiple subarrays within a bank to remain active simultaneously, improving DRAM parallelism. MASA’s memory controller uses a new subarray select command (SA_SEL) to designate the subarray that will serve the next read (RD) or write (WR) command. Since MASA exposes subarrays to the memory controller, it also provides the memory controller with a selective precharge

(PRE_SA) command to precharge an individual subarray without affecting other subarrays in the same bank.

Terracotta-based MASA. Terracotta implements MASA’s SA_SEL and PRE_SA commands using its custom command extensions. Each custom command requires a single payload to convey the subarray address. Terracotta tracks the currently selected subarray of each bank by utilizing the metadata table. We configure Terracotta’s controller primitives as follows. The trigger unit uses the tracked selected subarray to fire the two custom commands. It fires a subarray select when a scheduled request targets an active but non-selected subarray, and it fires a selective precharge when a scheduled request targets a different row in an already active subarray. The update unit updates the selected subarray in the metadata table on every subarray select or selective precharge. The action unit issues the custom SA_SEL and PRE_SA commands.

Results. Fig. 11 shows the performance of the Terracotta-based MASA implementation, a custom MASA implementation, and an ideal DRAM system where every subarray is modeled as a DRAM bank (“Ideal”), for single-core workloads in subplot (a) and four-core workload mixes in subplot (b). The x-axis shows the workload types with varying memory intensities and the y-axis shows speedup normalized to a baseline without subarray-level parallelism. In each workload type, the three bars from left to right show the mean performance of Terracotta-based MASA (labeled Terracotta-MASA), custom MASA (labeled MASA), and the ideal DRAM system (labeled “Ideal”), and error bars show the standard error of the mean across all the workloads of the workload type. *GMean* shows the geometric mean performance of each implementation across the workloads of all types.

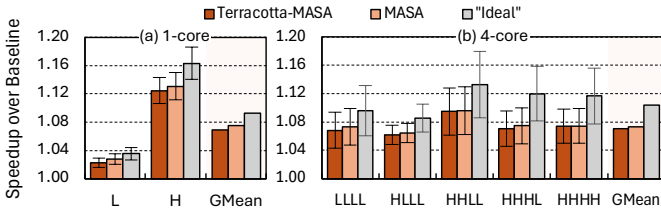


Figure 11: Performance of Terracotta-MASA, custom MASA, and an ideal DRAM system on single-core workloads and four-core workload mixes.

We make the following two observations. First, for both single-core workloads and four-core workload mixes, Terracotta-MASA closely matches the speedup of MASA over the baseline across all workload types. Terracotta-MASA’s geometric mean performance is only 0.22% (0.51%) lower than MASA for four-core workload mixes (single-core workloads), retaining 96.72% (92.66%) of MASA’s performance improvement over the baseline. Second, Terracotta-MASA captures a similar fraction of the “Ideal” DRAM system’s performance improvement as MASA. For four-core workload mixes (single-core workloads), Terracotta-MASA provides 68.3% (74.6%) of “Ideal”’s geometric mean performance improvement over the baseline, compared to 70.7% (80.5%) for custom MASA. We conclude that Terracotta can effectively enable the benefits of techniques that improve DRAM parallelism.

8.4. Reducing Memory Latency: ChargeCache

The memory controller enforces a fixed set of DRAM timing constraints (e.g., tRCD) that the manufacturer sets conservatively to guarantee correct operation for the slowest cell under worst-case conditions [283,284,286,379]. Since DRAM is heavily optimized for cost-per-bit, these timing constraints, and thus DRAM access latencies, have remained largely stagnant across DRAM generations [5,6,49,50,282–285].

ChargeCache. ChargeCache [279] reduces DRAM access latency by leveraging two observations. First, the cells of a row that was recently precharged remain in a “highly charged” state. Second, highly charged cells can be accessed faster than cells at a lower charge level, so an activation to such a row can use reduced timing constraints. ChargeCache’s memory controller tracks the addresses of recently precharged rows in a small controller-side table and serves subsequent activations to those rows with reduced timing constraints.

Terracotta-based ChargeCache. Terracotta uses its metadata table to track recently precharged rows. We configure Terracotta’s controller primitives as follows. The trigger unit performs a metadata lookup when an activation (ACT) is scheduled. On a metadata hit, it marks the request’s technique ID (*techID*) field to signal a faster access. The update unit performs two tasks. Upon a precharge (PRE), it inserts the open row’s address into the metadata table to track its highly charged state. Upon an ACT to a tracked row, it loads the relevant timers with the reduced timing constraints. The action unit is not used because ChargeCache requires no custom commands.

Results. Fig. 12 shows the performance of the Terracotta-based ChargeCache implementation, a custom ChargeCache implementation, and an ideal DRAM system where every activation uses reduced timing constraints, for single-core workloads in subplot (a) and four-core workload mixes in subplot (b). The x-axis shows the workload types with varying memory intensities and the y-axis shows speedup normalized to the baseline. In each workload type, the three bars from left to right show the mean performance of Terracotta-based ChargeCache (labeled Terracotta-ChargeCache), custom ChargeCache (labeled ChargeCache), and the ideal DRAM system (labeled “Low-Latency DRAM”), and error bars show the standard error of the mean across all the workloads of the workload type. *GMean* shows the geometric mean performance of each implementation across the workloads of all types.

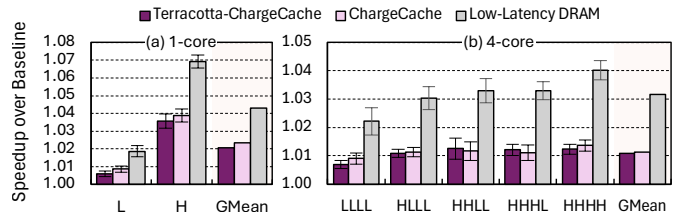


Figure 12: Performance of Terracotta-ChargeCache, custom ChargeCache, and a low-latency DRAM system on single-core workloads and four-core workload mixes.

We observe that although a custom ChargeCache provides modest performance improvements, Terracotta-ChargeCache retains 96.4% (87.7%) of these benefits, providing a geometric

mean speedup of 1.09% (2.04%) over the baseline for four-core workload mixes (single-core workloads). We conclude that Terracotta maintains most of the performance benefits of memory access latency reduction techniques such as ChargeCache despite its memory controller latency overheads.

8.5. Composing ChargeCache and MASA

With Terracotta, a system initially deployed with one technique can later adopt another technique supported by its existing hardware via a firmware update. We leverage Terracotta to compose ChargeCache (§8.4) and MASA (§8.3) into *ChargeCacheMASA* and show that Terracotta can retain the benefits of their composition. To configure ChargeCacheMASA, we configure the two techniques and also set the trigger unit’s C/A bus utilization filter threshold and trigger priorities (§6.2.2).

Results. Fig. 13 shows the performance of Terracotta-based and custom implementations of ChargeCache, MASA, and ChargeCacheMASA,⁹ for single-core workloads in subplot (a) and four-core workload mixes in subplot (b). The x-axis shows the workload types with varying memory intensities and the y-axis shows speedup normalized to the baseline. In each workload type, the three pairs of bars from left to right show ChargeCache, MASA, and ChargeCacheMASA. Within each pair, the left bar shows the mean performance of the Terracotta-based implementation (labeled Terracotta-{technique}) and the right bar shows the custom implementation (labeled {technique}), and error bars show the standard error of the mean across all the workloads of the workload type. *GMean* shows the geometric mean performance of each implementation across the workloads of all types.

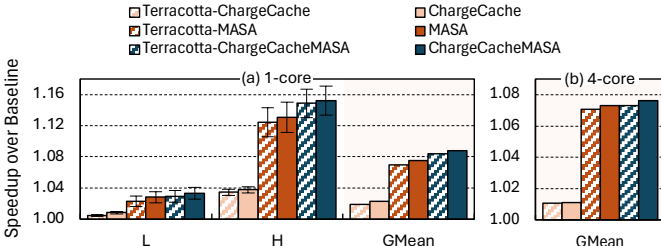


Figure 13: Performance of Terracotta-based and custom implementations of ChargeCache, MASA, and their combination (ChargeCacheMASA) on single-core workloads and four-core workload mixes with varying memory intensities.

We draw two key observations. First, the Terracotta implementation of ChargeCacheMASA provides performance benefits over the Terracotta implementations of both individual techniques. For single-core workloads, it provides speedups of 6.4% and 1.4% over Terracotta-ChargeCache and Terracotta-MASA, respectively. For four-core workload mixes, the corresponding speedups are 6.1% and 0.2%. The benefits of combining ChargeCache and MASA are *sub-additive*, especially for four-core workload mixes, because MASA keeps more rows activated, reducing the opportunities for ChargeCache to accelerate accesses to recently precharged rows. Sec-

⁹Among the configurations evaluated, we find the best performance by setting the C/A bus utilization filter threshold to 40% utilization (§6.2.2) and prioritizing MASA over ChargeCache when both remain eligible for a request (§6.2.2).

ond, Terracotta-ChargeCacheMASA closely matches the performance of ChargeCacheMASA, retaining 95.9% (95.8%) of its performance improvement over the baseline for four-core workload mixes (single-core workloads). We conclude that Terracotta preserves the benefits of combining techniques and enables a system to improve performance by adding additional techniques through firmware updates.

8.6. Overheads of Terracotta

8.6.1. Energy. We evaluate the DRAM energy consumption of the four techniques (§8.1–§8.4) and their composition (§8.5). Here, we report *only* the DRAM energy overhead introduced by Terracotta relative to custom implementations. Fig. 14 shows this overhead for each technique and their composition.

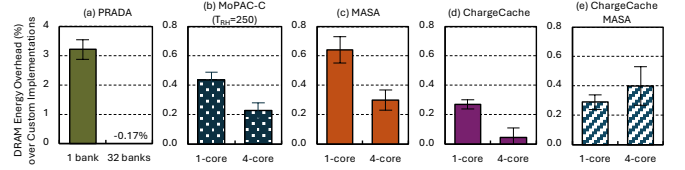


Figure 14: DRAM energy overhead (%) of Terracotta implementations over custom implementations. Note the different y-axis scales across subplots.

We observe that Terracotta-based PRADA consumes 3.2% more DRAM energy than custom PRADA when performing in-DRAM computation operations in 1 bank, but reduces average DRAM energy consumption by 0.17% when operating across 32 banks. We attribute this reduction to more favorable memory request scheduling arising from differences in controller and command latencies, which slightly shorten execution time. For the remaining techniques and their composition, the average DRAM energy overhead remains below 0.64% (0.40%) for single-core workloads (four-core workload mixes).

8.6.2. Memory Access Latency. We evaluate Terracotta’s impact on memory access latency by comparing the 99th-percentile memory access latency of Terracotta implementations with custom implementations. We evaluate MoPAC-C (at $T_{RH}=250$), MASA, and ChargeCache using four-core workload mixes, and PRADA using in-DRAM computation operations across 32 banks. We observe that Terracotta increases the 99th-percentile memory access latency by *only* 0.17%.

8.6.3. Area and Power. Our RTL synthesis results, obtained using the methodology in §6.5, show that Terracotta incurs modest area and power overheads over a design with custom controller support for the evaluated DRAM techniques. These additional area and power amount to *only* 0.03% of the die area and 0.56% of the thermal design power (TDP) of a high-end Intel Xeon processor [348]. These overheads constitute a one-time implementation cost that can be amortized across the techniques deployed over the processor’s lifetime.

8.6.4. Sensitivity to Controller Latency Overhead. Terracotta’s trigger unit adds a controller latency overhead of two cycles (§6.5). To examine how a higher controller latency affects performance, we sweep this overhead from two cycles to 10 cycles in increments of two cycles for the four techniques (§8.1–§8.4) and their composition (§8.5). Fig. 15 shows their

performance relative to their respective baselines. We evaluate PRADA using in-DRAM computation operations across 32 banks (subplot (a)) and the remaining techniques and their composition using four-core workload mixes (subplots (b)–(e)).

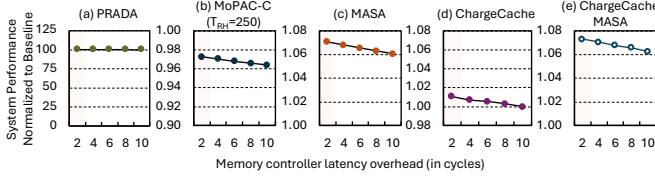


Figure 15: Performance sensitivity of Terracotta implementations to memory controller latency overhead. Note the different y-axis scales across subplots.

We draw two observations. First, PRADA’s performance remains unchanged with increase in memory controller latency overhead. We attribute this to the high bank-level parallelism exposed by our microbenchmarks, which perform independent in-DRAM computation operations across 32 banks. Second, the other techniques and their composition show a marginal performance decrease across the evaluated range of memory controller latency overhead. We conclude that the performance of Terracotta-based implementations is robust to higher controller latency overheads.

8.7. Terracotta with Reconfigurable Logic

§6 describes a concrete implementation of Terracotta using programmable tables. In this section, we evaluate the performance implications of an alternative Terracotta implementation that uses reconfigurable logic. Specifically, we implement Terracotta’s trigger and update units using 6×6 HyCUBE-like Coarse-Grained Reconfigurable Arrays (CGRAs) [352,353]. We retain programmable tables for the action primitive because issuing custom DRAM commands does *not* require additional flexibility. We map the trigger and update logic of each technique onto the CGRA using Morpher [380,381], a CGRA compiler, to determine the resulting trigger and update latencies, and incorporate them into Ramulator 2.0 [316–323].

Results. Fig. 16 shows the performance of the CGRA-based Terracotta implementation (labeled Terracotta-CGRA), the programmable table-based Terracotta implementation (labeled Terracotta-Tables), and a custom implementation (labeled Custom). We evaluate PRADA using in-DRAM computation operations across 32 banks (subplot (a)) and the remaining techniques and their composition using four-core workload mixes (subplots (b)–(e)). The y-axes show their speedup normalized to their respective baselines. In each subplot, the three bars from left to right show the geometric mean performance of

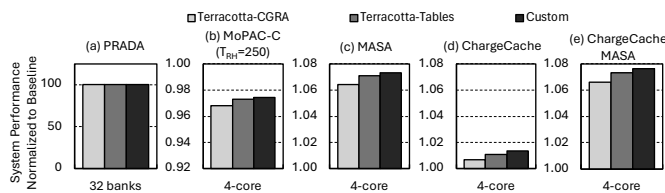


Figure 16: Performance of Terracotta-CGRA, Terracotta-Tables, and custom implementations for four techniques and their composition. Note the different y-axis scales across subplots.

Terracotta-CGRA, Terracotta-Tables, and Custom.

We observe that Terracotta-CGRA provides similar performance to custom implementations, with little or no performance loss relative to Terracotta-Tables. For PRADA, Terracotta-CGRA preserves nearly all the performance benefits of the custom implementation. For the remaining techniques and their composition, its performance is only 0.37–0.66% lower than Terracotta-Tables. We conclude that reconfigurable logic enables support for richer control flow while largely preserving the performance benefits of custom and programmable table-based Terracotta implementations.

9. Discussion

9.1. Extending Terracotta

We derive Terracotta’s primitives from an analysis of 32 techniques across seven categories. However, these primitives may *not* capture the requirements of all possible DRAM techniques. We discuss cases that may require extending Terracotta and, where possible, outline potential extensions. We leave the design and evaluation of these extensions to future work.

Multi-bank coordination. Terracotta partitions its trigger logic across banks (§5) and does *not* support coordination across them. Some Processing-near-Memory (PnM) techniques [29–31,53–60,62,65–68,73,382–387], which add logic *near* memory arrays to alleviate the data movement bottleneck [18,22–24,28–30,33,374], require such coordination. These techniques use a single command to perform the same operation across multiple banks [53–55,387] or all banks [56,57,60]. At the interface level, Terracotta’s custom command extensions can already encode operations on predefined groups of banks, including all banks, by allowing the DRAM-side command decoder to interpret the locality fields based on the command’s operation type. Targeting an arbitrary set of banks may require extending the locality primitive. At the controller level, one potential extension is to introduce channel- or bank group-level triggers to coordinate the firing of techniques across banks. Such a controller extension may also benefit memory scheduling [304,350,388] by coordinating memory requests across bank groups and banks to improve system performance and fairness.

DRAM feedback. Terracotta’s triggers, updates, and actions operate on the scheduled command and stored metadata, but do *not* directly observe the outcomes of DRAM operations. Techniques that use these outcomes to adapt DRAM operation therefore require additional support. Examples include techniques that use online profiling for retention-aware refresh [45,236,251,270] and access-time reduction [283,284,286]. Such feedback could also enable online profiling of read disturbance characteristics, including RowHammer thresholds, their variation across memory locations [199] and over time [41], and their dependence on temperature and row open time [37,144,389–391]. These profiles could enable future mitigations that adapt their preventive actions to the variation in read disturbance characteristics across chips, time, physical locations, and data patterns [392,393]. One potential extension is a feedback path that records profiling results in Terracotta’s

state, allowing triggers and updates to use these results to adapt subsequent DRAM operations.

9.2. Limitations of Terracotta

We identify Terracotta’s limitations under three categories.

Technique-specific DRAM modifications. Many DRAM techniques require modifications to the DRAM chip itself, in addition to the interface and controller support that Terracotta provides (§5). These modifications are specific to each technique. Supporting all of them in a single DRAM architecture could add circuitry that individual products might *not* need. Terracotta leaves these modifications to DRAM vendors to allow each vendor to implement the techniques they see fit and expose them through custom command extensions. This allows vendors to introduce technique-specific DRAM functionality while reducing the need for interface and memory controller modifications.

Technique-specific processor modifications. Some techniques [31,394–400] require changes to the core or the memory hierarchy to offload work to the DRAM system. These changes remain the system designer’s responsibility and fall outside the flexibility that Terracotta provides, although Terracotta can still support the parts of these techniques that need DRAM interface and controller support.

Implementation constraints. A given Terracotta implementation provides a fixed amount of metadata storage and a fixed number of timers (§5). A technique’s updates remain off the critical path only if its minimum timing constraint exceeds the metadata table read-modify-write latency (§6.5). Techniques that need more resources, or whose timing constraints are tighter than this latency, may require a controller redesign.

10. Related Work

To our knowledge, Terracotta is the first work that reduces the repeated interface and controller modifications needed to enable the adoption of DRAM techniques [8–12,34–48,69–299].

Self-Managing DRAM (SMD) [182,401] and Terracotta have different goals. SMD aims to transfer the responsibility for DRAM maintenance from the memory controller to the DRAM chip, enabling the DRAM chip to autonomously maintain itself. Its purpose is *not* to provide support for new DRAM techniques that require controller-side changes. In contrast, Terracotta aims to enable such DRAM techniques via a flexible DRAM interface and a programmable memory controller by reducing the need for repeated interface and controller modifications. As such, these frameworks are complementary and can be combined to enable more powerful innovations in the DRAM system [5,6,40,43,215,216,342] to mitigate the limitations of modern DRAM-based systems.

PARDIS [402] provides programmable control over address resolution, scheduling policies, and power management to improve system performance and reduce energy consumption. However, PARDIS *cannot* implement techniques that introduce new DRAM commands or maintain metadata in the memory controller. In contrast, Terracotta provides a flexible framework to enable techniques and commands beyond the standard, making the two approaches complementary.

FPGA-based DRAM infrastructures like DRAM Bender [403–405] and SoftMC [406,407] are instrumental to understanding DRAM robustness issues [36–38,40–42,44,45,114,199,214,270,271,408], understanding DRAM latency [49,50,283,284,379,409] and retention [45,46,251,262,270] characteristics, uncovering new DRAM functionality [70,71,80–83,90,98,100–102,230,299,410], and gaining insight into DRAM’s internal structure [51,251,271,379,389,390,408,409]. PiDRAM [113,375] and EasyDRAM [411,412] provide end-to-end system integration for some novel DRAM functionalities. These infrastructures expose functionality through existing DRAM commands. In contrast, Terracotta enables the deployment of diverse DRAM techniques through custom command extensions that let DRAM vendors define new commands within a standardized interface extension and a programmable memory controller that provides technique-specific support post-silicon fabrication.

11. Conclusion

We propose Terracotta, a new, flexible framework that reduces the repeated DRAM interface and memory controller modifications needed to adopt diverse DRAM techniques by capturing their similarities in DRAM commands and controller workflows into a set of configurable primitives. Terracotta consists of two components: (i) *custom command extensions* that enable DRAM vendors to define new commands within a single, standardized interface extension, and (ii) a *programmable memory controller* that system designers can easily and flexibly program to support new DRAM techniques post-silicon fabrication. We use Terracotta to implement four DRAM techniques across the domains of processing-using-DRAM, low-cost DRAM maintenance, subarray-level parallelism, and latency reduction. We show that Terracotta provides almost all of the performance benefits of their respective custom implementations, at low DRAM energy overheads and controller area and power overheads. We hope and expect that Terracotta will inspire academia and industry to develop innovative DRAM techniques and help overcome stagnation in their adoption, enabling large improvements in DRAM-based systems.

Acknowledgments

We thank the anonymous reviewers of MICRO SRC 2025, ISCA 2026, and MICRO 2026 for their feedback. We thank members of the SAFARI Research Group (especially Umut Başer and Vlad-Petru Nitu) for their constructive feedback and for providing a stimulating, intellectual, and scientific environment. We acknowledge the generous gift funding provided by our industrial partners (especially Google, Huawei, Intel, Microsoft), which has been instrumental in enabling the research we have been conducting on Processing-in-Memory and read disturbance in DRAM, in particular, and memory systems in general [3,4,7–18,30,36–38,42,44,47,78,115,116,202,237,303,321,322,405,413–416]. This work was in part supported by the Google Security and Privacy Research Award and the Microsoft Swiss Joint Research Center.

References

- [1] R. H. Dennard, "Field-Effect Transistor Memory," 1968, US Patent 3,387,286.
- [2] R. Dennard, F. Gaensslen, H.-N. Yu, V. Rideout, E. Bassous, and A. LeBlanc, "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions," *JSSC*, 1974.
- [3] O. Mutlu, "Memory Scaling: A Systems Architecture Perspective," in *IMW*, 2013.
- [4] O. Mutlu and L. Subramanian, "Research Problems and Opportunities in Memory Systems," *SUPERFRI*, 2014.
- [5] M. Patel, T. Shahroodi, A. Manglik, A. G. Yaglikci, A. Olgun, H. Luo, and O. Mutlu, "A Case for Transparent Reliability in DRAM Systems," arXiv:2204.10378 [cs.AR], 2022.
- [6] M. Patel, T. Shahroodi, A. Manglik, A. G. Yaglikci, A. Olgun, H. Luo, and O. Mutlu, "Rethinking the Producer-Consumer Relationship in Modern DRAM-Based Systems," *IEEE Access*, 2024.
- [7] O. Mutlu, "The RowHammer Problem and Other Issues We May Face as Memory Becomes Denser," in *DATE*, 2017.
- [8] O. Mutlu, A. Olgun, and I. E. Yuksel, "Memory-Centric Computing: Solving Computing's Memory Problem," in *IMW*, 2025.
- [9] O. Mutlu, A. Olgun, G. F. Oliveira, and I. E. Yuksel, "Memory-Centric Computing: Recent Advances in Processing-in-DRAM," in *IEDM*, 2024.
- [10] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A Modern Primer on Processing in Memory," in *Emerging Computing: From Devices to Systems – Looking Beyond Moore and Von Neumann*. Springer, 2022.
- [11] O. Mutlu, S. Ghose, J. Gómez-Luna, R. Ausavarungnirun, M. Sadrosadati, and G. F. Oliveira, "A Modern Primer on Processing in Memory," arXiv:2012.03112 [cs.AR], 2025.
- [12] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Processing Data Where It Makes Sense: Enabling In-Memory Computation," *MicPro*, 2019.
- [13] O. Mutlu, "RowHammer and Beyond," in *COSADE*, 2019.
- [14] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "Enabling Practical Processing in and Near Memory for Data-Intensive Computing," in *DAC*, 2019.
- [15] O. Mutlu, "Intelligent Architectures for Intelligent Computing Systems," in *DATE*, 2020.
- [16] O. Mutlu, A. Olgun, and A. G. Yaglikci, "Fundamentally Understanding and Solving RowHammer," in *ASP-DAC*, 2023.
- [17] I. E. Yuksel, F. N. Bostanci, A. Olgun, and O. Mutlu, "Memory-Centric Computing: Security Benefits and Challenges of Processing-in-DRAM," in *ICS Workshops*, 2026.
- [18] S. Ghose, A. Boroumand, J. S. Kim, J. Gómez-Luna, and O. Mutlu, "Processing-in-Memory: A Workload-Driven Perspective," *IBM JRD*, 2019.
- [19] S. Hong, "Memory Technology Trend and Future Challenges," in *IEDM*, 2010.
- [20] M. V. Wilkes, "The Memory Gap and the Future of High Performance Memories," *ACM SIGARCH Computer Architecture News*, 2001.
- [21] W. A. Wulf and S. A. McKee, "Hitting the Memory Wall: Implications of the Obvious," in *SIGARCH*, 1995.
- [22] S. Kanev, J. P. Darago, K. Hazelwood, P. Ranganathan, T. Moseley, G.-Y. Wei, and D. Brooks, "Profiling a Warehouse-scale Computer," in *ISCA*, 2015.
- [23] J. Dean and L. A. Barroso, "The Tail at Scale," *CACM*, 2013.
- [24] M. Ferdman, A. Adileh, O. Kocerber, S. Volos, M. Alisafae, D. Jevdjic, C. Kaynak, A. D. Popescu, A. Ailamaki, and B. Falsafi, "Clearing the Clouds: A Study of Emerging Scale-Out Workloads on Modern Hardware," in *ASPLOS*, 2012.
- [25] S. Ghose, A. G. Yaglikci, R. Gupta, D. Lee, K. Kudrolli, W. X. Liu, H. Hassan, K. K. Chang, N. Chatterjee, A. Agrawal *et al.*, "What Your DRAM Power Models are Not Telling You: Lessons from a Detailed Experimental Study," *SIGMETRICS*, 2018.
- [26] T. Vogelsang, "Understanding the Energy Consumption of Dynamic Random Access Memories," in *MICRO*, 2010.
- [27] S. Wang and E. Ipek, "Reducing Data Movement Energy via Online Data Clustering and Encoding," in *MICRO*, 2016.
- [28] D. Pandiyan and C.-J. Wu, "Quantifying the Energy Cost of Data Movement for Emerging Smart Phone Workloads on Mobile Platforms," in *IISWC*, 2014.
- [29] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing," in *ISCA*, 2015.
- [30] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "Retrospective: A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing," *Retrospective Issue for ISCA-50*, 2023.
- [31] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-Enabled Instructions: A Low-overhead, Locality-aware Processing-in-Memory Architecture," in *ISCA*, 2015.
- [32] S. Koppula, L. Orosa, A. G. Yaglikci, R. Azizi, T. Shahroodi, K. Kanellopoulos, and O. Mutlu, "EDEN: Enabling Energy-Efficient, High-Performance Deep Neural Network Inference Using Approximate DRAM," in *MICRO*, 2019.
- [33] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungnirun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan, and O. Mutlu, "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks," in *ASPLOS*, 2018.
- [34] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungnirun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch *et al.*, "RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization," in *MICRO*, 2013.
- [35] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology," in *MICRO*, 2017.
- [36] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," in *ISCA*, 2014.
- [37] H. Luo, A. Olgun, A. G. Yaglikci, Y. C. Tuğrul, S. Rhyner, M. B. Cavlak, J. Lindegger, M. Sadrosadati, and O. Mutlu, "RowPress: Amplifying Read Disturbance in Modern DRAM Chips," in *ISCA*, 2023.
- [38] I. E. Yuksel, A. Olgun, N. Bostanci, H. Luo, A. G. Yaglikci, and O. Mutlu, "ColumnDisturb: Understanding Column-based Read Disturbance in Real DRAM Chips and Implications for Future Systems," in *MICRO*, 2025.
- [39] O. Canpolat, A. G. Yaglikci, G. F. Oliveira, A. Olgun, N. Bostanci, I. E. Yuksel, H. Luo, O. Ergin, and O. Mutlu, "Chronus: Understanding and Securing the Cutting-Edge Industry Solutions to DRAM Read Disturbance," in *HPCA*, 2025.
- [40] J. S. Kim, M. Patel, A. G. Yaglikci, H. Hassan, R. Azizi, L. Orosa, and O. Mutlu, "Revisiting RowHammer: An Experimental Analysis of Modern Devices and Mitigation Techniques," in *ISCA*, 2020.
- [41] A. Olgun, F. N. Bostanci, I. E. Yuksel, O. Canpolat, H. Luo, G. F. Oliveira, A. G. Yaglikci, M. Patel, and O. Mutlu, "Variable Read Disturbance: An Experimental Analysis of Temporal Variation in DRAM Read Disturbance," in *HPCA*, 2025.
- [42] H. Luo, İ. E. Yuksel, A. Olgun, N. Bostanci, O. Ececiş, A. G. Yaglikci, and O. Mutlu, "DejaVu: Why You Should Write to Your DRAM Rows Twice, Carefully," in *ISCA*, 2026.
- [43] P. Frigo, E. Vannacci, H. Hassan, V. van der Veen, O. Mutlu, C. Giuffrida, H. Bos, and K. Razavi, "TRRepass: Exploiting the Many Sides of Target Row Refresh," in *S&P*, 2020.
- [44] I. E. Yuksel, A. Sood, A. Olgun, O. Canpolat, H. Luo, N. Bostanci, M. Sadrosadati, G. Yaglikci, and O. Mutlu, "PuDHammer: Experimental Analysis of Read Disturbance Effects of Processing-using-DRAM in Real DRAM Chips," in *ISCA*, 2025.
- [45] M. K. Qureshi, D.-H. Kim, S. Khan, P. J. Nair, and O. Mutlu, "AVATAR: A Variable-Retention-Time (VRT) Aware Refresh for DRAM Systems," in *DSN*, 2015.
- [46] J. Liu, B. Jaiyen, Y. Kim, C. Wilkerson, and O. Mutlu, "An Experimental Study of Data Retention Behavior in Modern DRAM Devices," in *ISCA*, 2013.
- [47] O. Mutlu, "Retrospective: An Experimental Study of Data Retention Behavior in Modern DRAM Devices: Implications for Retention Time Profiling Mechanisms," *Retrospective Issue for ISCA-50*, 2023.
- [48] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, "A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM," in *ISCA*, 2012.
- [49] K. K. Chang, "Understanding and Improving Latency of DRAM-Based Memory Systems," Ph.D. dissertation, Carnegie Mellon University, 2017.
- [50] D. Lee, "Reducing DRAM Latency at Low Cost by Exploiting Heterogeneity," Ph.D. dissertation, Carnegie Mellon University, 2016.
- [51] M. Patel, J. S. Kim, H. Hassan, and O. Mutlu, "Understanding and Modeling On-Die Error Correction in Modern DRAM: An Experimental Study Using Real Devices," in *DSN*, 2019.
- [52] A. Boroumand, S. Ghose, B. Akin, R. Narayanaswami, G. F. Oliveira, X. Ma, E. Shiu, and O. Mutlu, "Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks," in *PACT*, 2021.
- [53] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, J. Chun, and Sk hynix, "System Architecture and Software Stack for GDDR6-AiM," in *Hot Chips*, 2022.
- [54] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning," in *MICRO*, 2020.
- [55] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A 1ybm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications," in *ISSCC*, 2022.
- [56] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, O. Seongil, H.-S. Yu, H. Lee, S. Y. Kim, Y. Cho, J. G. Kim, J. Choi, H.-S. Shin, J. Kim, B. Phuah, H. Kim, M. J. Song, A. Choi, D. Kim, S. Kim, E.-B. Kim, D. Wang, S. Kang, Y. Ro, S. Seo, J. Song, J. Yoon, K. Sohn, and N. S. Kim, "A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications," in *ISSCC*, 2021.
- [57] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product," in *ISCA*, 2021.
- [58] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System," in *ASPLOS*, 2025.
- [59] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference," in *ASPLOS*, 2025.
- [60] W. Kim, Y. Lee, Y. Kim, J. Hwang, S. Oh, J. Jung, A. Huseynov, W. G. Park, C. H. Park, D. Mahajan, and J. Park, "Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving," in *MICRO*, 2025.
- [61] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference," in *ASPLOS*, 2024.
- [62] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *ASPLOS*, 2024.
- [63] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, "SpecPIM: Accelerating Speculative Inference on PIM-Enabled System via Architecture-Dataflow Co-Exploration," in *ASPLOS*, 2024.

- [64] P. J. Nair, R. Hadidi, S. Ganesh, S. Kodge, S. Rathore, N. Thanawala, N. Reddy, G. Saharia, V. Patankar, A. Tiruvur, N. Kurella, and S. Bhoja, "Early Silicon of Raptor: The First 3D-DRAM Accelerator for Generative Inference," in *ISCA*, 2026.
- [65] H. Kal, C. Yoo, and W. W. Ro, "AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory," in *MICRO*, 2023.
- [66] S. Yu, H. Kim, K. Jeun, S. Hwang, S. Cho, and E. Lee, "ComPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration," in *MICRO*, 2025.
- [67] H. Shin, D. Kim, E. Park, S. Park, Y. Park, and S. Yoo, "McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM," *TCADICS*, 2018.
- [68] S. Cho, H. Choi, E. Park, H. Shin, and S. Yoo, "McDRAM v2: In-Dynamic Random Access Memory Systolic Array Accelerator to Address the Large Model Problem in Deep Neural Networks on the Edge," *IEEE Access*, 2020.
- [69] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Buddy-RAM: Improving the Performance and Efficiency of Bulk Bitwise Operations Using DRAM," arXiv:1605.06483 [cs.AR], 2016.
- [70] I. E. Yüksel, Y. C. Tuğrul, A. Olgun, F. N. Bostancı, A. G. Yağlıkçı, G. F. Oliveira, H. Luo, J. Gómez-Luna, M. Sadrosadati, and O. Mutlu, "Functionally-Complete Boolean Logic in Real DRAM Chips: Experimental Characterization and Analysis," in *HPCA*, 2024.
- [71] I. E. Yüksel, Y. C. Tuğrul, F. N. Bostancı, G. F. Oliveira, A. G. Yağlıkçı, A. Olgun, M. Soysal, H. Luo, J. Gómez-Luna, M. Sadrosadati, and O. Mutlu, "Simultaneous Many-Row Activation in Off-the-Shelf DRAM Chips: Experimental Characterization and Analysis," in *DSN*, 2024.
- [72] X. Xin, Y. Zhang, and J. Yang, "ELP2IM: Efficient and Low Power Bitwise Operation Processing in DRAM," in *HPCA*, 2020.
- [73] M. Besta, R. Kanakagiri, G. Kwasniewski, R. Ausavarungnirun, J. Beránek, K. Kanellopoulos, K. Janda, Z. Vonnarburg-Shmaria, L. Gianinazzi, I. Stefan *et al.*, "SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems," in *MICRO*, 2021.
- [74] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "DRISA: A DRAM-Based Reconfigurable In-Situ Accelerator," in *MICRO*, 2017.
- [75] S. Li, A. O. Glova, X. Hu, P. Gu, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "SCOPE: A Stochastic Computing Engine for DRAM-Based In-Situ Accelerator," in *MICRO*, 2018.
- [76] Q. Deng, L. Jiang, Y. Zhang, M. Zhang, and J. Yang, "DrAcc: A DRAM Based Accelerator for Accurate CNN Inference," in *DAC*, 2018.
- [77] S. Angizi and D. Fan, "GraphiDe: A Graph Processing Accelerator Leveraging In-DRAM-Computing," in *GLSVLSI*, 2019.
- [78] G. F. Oliveira, J. Gómez-Luna, S. Ghose, A. Boroumand, and O. Mutlu, "Accelerating Neural Network Inference with Processing-in-DRAM: From the Edge to the Cloud," *IEEE Micro*, 2022.
- [79] N. Hajinazar, G. F. Oliveira, S. Gregorio, J. D. Ferreira, N. Mansouri Ghiasi, M. Patel, M. Alser, S. Ghose, J. Gómez-Luna, and O. Mutlu, "SIMDRAM: A Framework for Bit-Serial SIMD Processing Using DRAM," in *ASPLOS*, 2021.
- [80] J. S. Kim, M. Patel, H. Hassan, L. Orosa, and O. Mutlu, "D-RaNGe: Using Commodity DRAM Devices to Generate True Random Numbers with Low Latency and High Throughput," in *HPCA*, 2019.
- [81] F. Gao, G. Tziatzoulis, and D. Wentzlaff, "ComputeDRAM: In-Memory Compute Using Off-the-Shelf DRAMs," in *MICRO*, 2019.
- [82] F. Gao, G. Tziatzoulis, and D. Wentzlaff, "FracDRAM: Fractional Values in Off-the-Shelf DRAM," in *MICRO*, 2022.
- [83] A. Olgun, M. Patel, A. G. Yağlıkçı, H. Luo, J. S. Kim, F. N. Bostancı, N. Vijaykumar, O. Ergin, and O. Mutlu, "QUAC-TRNG: High-Throughput True Random Number Generation Using Quadruple Row Activation in Commodity DRAM Chips," in *ISCA*, 2021.
- [84] T. Kubo, M. Usui, T. Nagatani, D. Tokuda, L. Qu, T. Cao, and S. Takamaeda-Yamazaki, "Bulk Bitwise Accumulation in Commercial DRAM," in *MLNCP*, 2024.
- [85] I. E. Yüksel, Y. C. Tuğrul, F. N. Bostancı, A. G. Yağlıkçı, A. Olgun, G. F. Oliveira, M. Soysal, H. Luo, J. Gómez-Luna, M. Sadrosadati, and O. Mutlu, "PULSAR: Simultaneous Many-Row Activation for Reliable and High-Performance Computing in Off-the-Shelf DRAM Chips," arXiv:2312.02880 [cs.AR], 2024.
- [86] R. Nadig, V. Arulchelvan, M. Kabra, H. Gupta, R. Bera, N. M. Ghiasi, N. Rao, Q. Jiang, A. K. Kakolyris, Y. Liang, M. Sadrosadati, and O. Mutlu, "Conduit: Programmer-Transparent Near-Data Processing Using Multiple Compute-Capable Resources in Solid State Drives," in *HPCA*, 2026.
- [87] F. N. Bostancı, A. Olgun, L. Orosa, A. G. Yağlıkçı, J. S. Kim, H. Hassan, O. Ergin, and O. Mutlu, "DR-STRaNGe: End-to-End System Design for DRAM-based True Random Number Generators," in *HPCA*, 2022.
- [88] M. F. Ali, A. Jaiswal, and K. Roy, "In-Memory Low-Cost Bit-Serial Addition Using Commodity DRAM Technology," *TCSI*, 2020.
- [89] Z. Jahshan and L. Yavits, "MajorK: Majority Based Kmer Matching in Commodity DRAM," *CAL*, 2024.
- [90] E. Garzón, A. Fish, and L. Yavits, "CADM: Content Addressable Commodity Off-the-Shelf DRAM-based Genome Classifier," *Journal of Systems Architecture*, 2026.
- [91] D. Yaron, B. Wolfson, Z. Jahshan, A. Fish, and L. Yavits, "BinDRAM: Binary Neural Network on Unmodified Commodity DRAM," *Future Generation Computing Systems*, 2026.
- [92] S. Angizi and D. Fan, "ReDRAM: A Reconfigurable Processing-in-DRAM Platform for Accelerating Bulk Bit-Wise Operations," in *ICCAD*, 2019.
- [93] R. Zhou, A. Roohi, D. Misra, and S. Angizi, "ReD-LUT: Reconfigurable In-DRAM LUTs Enabling Massive Parallel Computation," in *ICCAD*, 2022.
- [94] S. Afifi, I. Thakkar, and S. Pasricha, "ARTEMIS: A Mixed Analog-Stochastic In-DRAM Accelerator for Transformer Neural Networks," *TCAD*, 2024.
- [95] J. P. C. de Lima, B. Morris, A. A. Khan, J. Castrillon, and A. K. Jones, "Count2Multiply: Reliable In-Memory High-Radix Counting," in *HPCA*, 2026.
- [96] F. Li, R. Zhu, H. Li, D. Wu, and X. Xin, "PIM-SUM: Fast and Reliable In-Memory Summation for Recommendation Systems," in *ICCD*, 2025.
- [97] T. Kubo, D. Tokuda, T. Nagatani, M. Usui, L. Qu, T. Cao, and S. Takamaeda-Yamazaki, "MVDram: Enabling GeMV Execution in Unmodified DRAM for Low-Bit LLM Acceleration," arXiv:2503.23817 [cs.AR], 2025.
- [98] T. Kubo, D. Tokuda, L. Qu, T. Cao, and S. Takamaeda-Yamazaki, "PUDTune: Multi-Level Charging for High-Precision Calibration in Processing-Using-DRAM," *CAL*, 2025.
- [99] R. Zhou, S. Tabrizchi, A. Roohi, and S. Angizi, "LT-PIM: An LUT-Based Processing-in-DRAM Architecture With RowHammer Self-Tracking," *IEEE CAL*, 2022.
- [100] D. Tokuda, T. Kubo, I. E. Yüksel, A. Olgun, H. Luo, T. Nagatani, G. F. Oliveira, A. G. Yağlıkçı, M. Sadrosadati, O. Mutlu, and S. Takamaeda-Yamazaki, "Clutch: High Performance Vector-Scalar Comparison Using DRAM via Chunked Temporal Coding," in *ICS*, 2026.
- [101] J. S. Kim, M. Patel, H. Hassan, and O. Mutlu, "The DRAM Latency PUF: Quickly Evaluating Physical Unclonable Functions by Exploiting the Latency-Reliability Tradeoff in Modern Commodity DRAM Devices," in *HPCA*, 2018.
- [102] U. Baser, I. E. Yüksel, F. N. Bostancı, K. Sgouras, A. Olgun, E. H. Demirli, Z. Yue, H. Songara, O. Ergin, and O. Mutlu, "In-DRAM Signature Generation Using Simultaneous Multiple-Row Activation: An Experimental Study of Off-The-Shelf DRAM Chips," in *DSN Disrupt*, 2026.
- [103] M. A. Soleimani, N. Rohbani, A. C. Kestelman, O. Unsal, and H. Sarbazi-Azad, "WISEDRAM: A Reliable Bitwise In-DRAM Accelerator," in *DAC*, 2025.
- [104] H. Shin, R. Park, and J. W. Lee, "A Processing-using-Memory Architecture for Commodity DRAM Devices with Enhanced Compatibility and Reliability," in *ICCAD*, 2024.
- [105] H. Shin, "A Processing-in-Memory Architecture for Commodity DRAM Devices with Enhanced Reliability and Compatibility," Ph.D. dissertation, Seoul National University, 2023.
- [106] G. F. Oliveira, A. Olgun, A. G. Yağlıkçı, F. N. Bostancı, J. Gómez-Luna, S. Ghose, and O. Mutlu, "MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing," in *HPCA*, 2024.
- [107] J. D. Ferreira, G. Falcao, J. Gómez-Luna, M. Alser, L. Orosa, M. Sadrosadati, J. S. Kim, G. F. Oliveira, T. Shahroodi, A. Nori *et al.*, "pLUTO: Enabling Massively Parallel Computation in DRAM via Lookup Tables," in *MICRO*, 2022.
- [108] Q. Deng, Y. Zhang, M. Zhang, and J. Yang, "LAcc: Exploiting Lookup Table-based Fast and Accurate Vector Multiplication in DRAM-based CNN Accelerator," in *DAC*, 2019.
- [109] G. F. Oliveira, M. Kabra, Y. Guo, K. Chen, A. G. Yağlıkçı, M. Soysal, M. Sadrosadati, J. Olivares Bueno, S. Ghose, J. Gómez-Luna, and O. Mutlu, "Proteus: Achieving High-Performance Processing-Using-DRAM with Dynamic Bit-Precision, Adaptive Data Representation, and Flexible Arithmetic," in *ICS*, 2025.
- [110] V. Seshadri, K. Hsieh, A. Boroumand, D. Lee, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Fast Bulk Bitwise AND and OR in DRAM," *CAL*, 2015.
- [111] V. Seshadri and O. Mutlu, "The Processing Using Memory Paradigm: In-DRAM Bulk Copy, Initialization, Bitwise AND and OR," arXiv:1610.09603 [cs.AR], 2016.
- [112] V. Seshadri and O. Mutlu, "In-DRAM Bulk Bitwise Execution Engine," arXiv:1905.09822 [cs.AR], 2019.
- [113] A. Olgun, J. Gómez-Luna, K. Kanellopoulos, B. Salami, H. Hassan, O. Ergin, and O. Mutlu, "PiDRAM: A Holistic End-to-end FPGA-based Framework for Processing-in-DRAM," *TACO*, 2022.
- [114] D. Tokuda, I. E. Yüksel, T. Kubo, A. Olgun, H. Luo, N. Bostancı, J. Wang, A. G. Yağlıkçı, S. Takamaeda-Yamazaki, and O. Mutlu, "PuDghost: Experimental Analysis of Computation Result Corruption in Processing-using-DRAM Operations on Real DRAM Chips and Implications for Future Systems," in *ISCA*, 2026.
- [115] O. Mutlu and J. Kim, "RowHammer: A Retrospective," *IEEE TCAD Special Issue on Top Picks in Hardware and Embedded Security*, 2019.
- [116] O. Mutlu, "Retrospective: Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," *Retrospective Issue for ISCA-50*, 2023.
- [117] H. Hassan, Y. C. Tuğrul, J. S. Kim, V. van der Veen, K. Razavi, and O. Mutlu, "Uncovering In-DRAM RowHammer Protection Mechanisms: A New Methodology, Custom RowHammer Patterns, and Implications," in *MICRO*, 2021.
- [118] M. Marazzi, P. Jattke, F. Solt, and K. Razavi, "ProTRR: Principled yet Optimal In-DRAM Target Row Refresh," in *S&P*, 2022.
- [119] F. Brasser, L. Davi, D. Gens, C. Liebchen, and A.-R. Sadeghi, "Can't Touch This: Software-Only Mitigation Against Rowhammer Attacks Targeting Kernel Memory," in *USENIX Security*, 2017.
- [120] A. Chakraborty, M. Alam, and D. Mukhopadhyay, "Deep Learning Based Diagnostics for Rowhammer Protection of DRAM Chips," in *ATS*, 2019.
- [121] M. Son, H. Park, J. Ahn, and S. Yoo, "Making DRAM Stronger Against Row Hammering," in *DAC*, 2017.
- [122] H. Gomez, A. Amaya, and E. Roa, "DRAM Row-Hammer Attack Reduction Using Dummy Cells," in *NORCAS*, 2016.
- [123] R. K. Konoth, M. Oliverio, A. Tatar, D. Andriesse, H. Bos, C. Giuffrida, and K. Razavi, "ZebRAM: Comprehensive and Compatible Software Protection Against Rowhammer Attacks," in *OSDI*, 2018.
- [124] M. Kim, J. Choi, H. Kim, and H.-J. Lee, "An Effective DRAM Address Remapping for Mitigating Rowhammer Errors," *TC*, 2019.
- [125] X.-C. Wu, T. Sherwood, F. T. Chong, and Y. Li, "Protecting Page Tables from RowHammer Attacks Using Monotonic Pointers in DRAM True-Cells," in *ASPLOS*, 2019.

- [126] Z. B. Aweke, S. F. Yitbarek, R. Qiao, R. Das, M. Hicks, Y. Oren, and T. Austin, "ANVIL: Software-Based Protection Against Next-Generation Rowhammer Attacks," in *ASPLOS*, 2016.
- [127] Z. Zhang, Y. Cheng, M. Wang, W. He, W. Wang, S. Nepal, Y. Gao, K. Li, Z. Wang, and C. Wu, "SoftTRR: Protect Page Tables against Rowhammer Attacks Using Software-only Target Row Refresh," in *USENIX ATC*, 2022.
- [128] V. van der Veen, M. Lindorfer, Y. Fratanonio, H. P. Pillai, G. Vigna, C. Kruegel, H. Bos, and K. Razavi, "GuardION: Practical Mitigation of DMA-Based Rowhammer Attacks on ARM," in *DIMVA*, 2018.
- [129] K. Loughlin, S. Saroiu, A. Wolman, Y. A. Manerkar, and B. Kasicki, "MOESI-Prime: Preventing Coherence-Induced Hammering in Commodity Workloads," in *ISCA*, 2022.
- [130] S. Enomoto, H. Kuzuno, and H. Yamada, "Efficient Protection Mechanism for CPU Cache Flush Instruction Based Attacks," *IEICE TIS*, 2022.
- [131] S. M. Ajorpez, D. Moghimi, J. N. Collins, G. Pokam, N. Abu-Ghazaleh, and D. Tullsen, "EVAX: Towards a Practical, Pro-active & Adaptive Architecture for High Performance & Security," in *MICRO*, 2022.
- [132] A. Naseredini, M. Berger, M. Sammartino, and S. Xiong, "ALARM: Active Learning of Rowhammer Mitigations," <https://users.sussex.ac.uk/~mfb21/rh-draft.pdf>, 2022.
- [133] B. K. Joardar, T. K. Bletsch, and K. Chakrabarty, "Machine Learning-based Rowhammer Mitigation," *TCAD*, 2022.
- [134] Z. Zhang, Z. Zhan, D. Balasubramanian, B. Li, P. Volgyesi, and X. Koutsoukos, "Leveraging EM Side-Channel Information to Detect Rowhammer Attacks," in *S&P*, 2020.
- [135] I. Kang, E. Lee, and J. H. Ahn, "CAT-TWO: Counter-Based Adaptive Tree, Time Window Optimized for DRAM Row-Hammer Prevention," *IEEE Access*, 2020.
- [136] T. Bennett, S. Saroiu, A. Wolman, and L. Cojocar, "Panopticon: A Complete In-DRAM Rowhammer Mitigation," in *DRAMSec*, 2021.
- [137] A. G. Yağlıkçı, J. S. Kim, F. Devaux, and O. Mutlu, "Security Analysis of the Silver Bullet Technique for RowHammer Prevention," arXiv:2106.07084 [cs.CR], 2021.
- [138] J.-W. Han, J. Kim, D. Beery, K. D. Bozdag, P. Cuevas, A. Levi, I. Tain, K. Tran, A. J. Walker, S. V. Palayam, A. Arreghini, A. Furnémont, and M. Meyyappan, "Surround Gate Transistor With Epitaxially Grown Si Pillar and Simulation Study on Soft Error and Rowhammer Tolerance for DRAM," *IEEE TED*, 2021.
- [139] S. Saroiu and A. Wolman, "How to Configure Row-Sampling-Based Rowhammer Defenses," *DRAMSec*, 2022.
- [140] J. H. Park, S. Y. Kim, D. Y. Kim, G. Kim, J. W. Park, S. Yoo, Y.-W. Lee, and M. J. Lee, "RowHammer Reduction Using a Buried Insulator in a Buried Channel Array Transistor," *IEEE TED*, 2022.
- [141] M. Wi, J. Park, S. Ko, M. J. Kim, N. S. Kim, E. Lee, and J. H. Ahn, "SHADOW: Preventing Row Hammer in DRAM with Intra-Subarray Row Shuffling," in *HPCA*, 2023.
- [142] C. Bock, F. Brasser, D. Gens, C. Liebchen, and A. R. Sadeghi, "RIP-RH: Preventing Rowhammer-Based Inter-Process Attacks," in *ASIA-CCS*, 2019.
- [143] C. Gude Ramarao, K. T. Kumar, G. Ujjinappa, and B. V. D. Naidu, "Defending SoCs with FPGAs from Rowhammer Attacks," *Material Science*, 2023.
- [144] L. Orosa, A. G. Yağlıkçı, H. Luo, A. Olgun, J. Park, H. Hassan, M. Patel, J. S. Kim, and O. Mutlu, "A Deeper Look into RowHammer's Sensitivities: Experimental Analysis of Real DRAM Chips and Implications on Future Attacks and Defenses," in *MICRO*, 2021.
- [145] M. Marazzi, F. Solt, P. Jattke, K. Takashi, and K. Razavi, "REGA: Scalable Rowhammer Mitigation with Refresh-Generating Activations," in *S&P*, 2023.
- [146] K. Loughlin, S. Saroiu, A. Wolman, and B. Kasicki, "Stop! Hammer Time: Rethinking Our Approach to Rowhammer Mitigations," in *HotOS*, 2021.
- [147] B. K. Joardar, T. K. Bletsch, and K. Chakrabarty, "Learning to Mitigate RowHammer Attacks," in *DATE*, 2022.
- [148] H. Taneja, A. Hajiabadi, M. Marazzi, K. Razavi, and M. Qureshi, "MIRZA: Efficiently Mitigating Rowhammer with Randomization and ALERT," in *HPCA*, 2026.
- [149] M. Qureshi and S. Qazi, "MOAT: Securely Mitigating Rowhammer with Per-Row Activation Counters," in *ASPLOS*, 2025.
- [150] H. Taneja and M. K. Qureshi, "DREAM: Enabling Low-Overhead Rowhammer Mitigation via Directed Refresh Management," in *ISCA*, 2025.
- [151] C. S. Lin, J. Woo, P. J. Nair, and G. Saileshwar, "CnC-PRAC: Coalesce, Not Cache, Per Row Activation Counts for an Efficient in-DRAM Rowhammer Mitigation," in *DRAMSec*, 2025.
- [152] A. Jaleel, G. Saileshwar, S. W. Keckler, and M. Qureshi, "PrIDE: Achieving Secure Rowhammer Mitigation with Low-Cost In-DRAM Trackers," in *ISCA*, 2024.
- [153] M. Qureshi, S. Qazi, and A. Jaleel, "MINT: Securely Mitigating Rowhammer with a Minimalist in-DRAM Tracker," in *MICRO*, 2024.
- [154] R. Wu, M. Zhang, Y. Zhou, C. Xie, and F. Wu, "APT: Securing Against DRAM Read Disturbance via Adaptive Probabilistic In-DRAM Trackers," in *ASPLOS*, 2026.
- [155] M. K. Qureshi, "SALT: Track-and-Mitigate Subarrays, Not Rows, for Blast-Radius-Free Rowhammer Defense," in *HPCA*, 2026.
- [156] M. Qureshi, "CODA: How to Mitigate ColumnDisturb for (Almost) Free?" arXiv:2607.13505 [cs.CR], 2026.
- [157] H. Taneja and M. Qureshi, "RogueRFM: Attacking Refresh Management for Covert-Channel and Denial-of-Service," arXiv:2501.06646 [cs.CR], 2025.
- [158] M. Qureshi, "AutoRFM: Scaling Low-Cost in-DRAM Trackers to Ultra-Low Rowhammer Thresholds," in *HPCA*, 2025.
- [159] Z. Lang, P. Jattke, M. Marazzi, and K. Razavi, "BLASTER: Characterizing the Blast Radius of Rowhammer," in *DRAMSec*, 2023.
- [160] S.-L. Lu, J. Woo, P. J. Nair, and M. Qureshi, "Counterpoint: One-Hot Counting for PRAC-Based RowHammer Mitigation," in *DRAMSec*, 2025.
- [161] J. Woo, S. C. Lin, P. J. Nair, A. Jaleel, and G. Saileshwar, "QPRAC: Towards Secure and Practical PRAC-based Rowhammer Mitigation Using Priority Queues," in *HPCA*, 2025.
- [162] M. Wi, Y. Yoo, Y. Kim, J. Shin, J. Kim, Y. Ryu, S. Gorgin, J. H. Ahn, and J. Kim, "RowArmor: Efficient and Comprehensive Protection Against DRAM Disturbance Attacks," in *ASPLOS*, 2026.
- [163] A. Alajmi and H. Yang, "Dynamic Rowhammer Threshold Management: Temperature-Aware Threshold for In-DRAM Defenses," in *DRAMSec*, 2026.
- [164] F. Wang, Y. Xiao, and C. Yagemann, "FlipShield: Self-Improving Software Mitigation Against Rowhammer," in *DRAMSec*, 2026.
- [165] M. K. Qureshi, "elppiR: Efficiently Protecting PRAC Against Ripple Attacks," in *DRAMSec*, 2026.
- [166] S. Hong, D. Kim, J. Lee, R. Oh, C. Yoo, S. Hwang, and J. Lee, "DSAC: Low-Cost Rowhammer Mitigation Using In-DRAM Stochastic and Approximate Counting Algorithm," arXiv:2302.03591 [cs.CR], 2023.
- [167] H. Du, Y. Yang, S. Chen, and Y. Kang, "DIVIDE: Efficient RowHammer Defense via In-DRAM Cache-Based Hot Data Isolation," *TC*, 2025.
- [168] A. Saxena, W. Wang, and A. Daglis, "Citadel: Rethinking Memory Allocation to Safeguard Against Inter-Domain Rowhammer Exploits," in *MICRO*, 2025.
- [169] P. Jattke, M. Marazzi, F. Solt, M. Wipfli, S. Gloor, and K. Razavi, "McSee: Evaluating Advanced Rowhammer Attacks and Defenses via Automated DRAM Traffic Analysis," in *Security*, 2025.
- [170] F. N. Bostancı, K. Kanellopoulos, A. Olgun, A. G. Yağlıkçı, İ. Emir Yüksel, N. M. Ghiasi, Z. Bingöl, M. Sadrosadati, and O. Mutlu, "Revisiting Main Memory-Based Covert and Side Channel Attacks in the Context of Processing-in-Memory," in *DSN*, 2025.
- [171] M. J. Kim, S. Baek, J. Kim, H. Nam, N. S. Kim, and J. H. Ahn, "SoK: Systematizing a Decade of Architectural Rowhammer Defenses Through the Lens of Streaming Algorithms," in *S&P*, 2026.
- [172] J. Wang, H. Luo, A. Olgun, İ. E. Yüksel, A. G. Yağlıkçı, Y. Liang, F. N. Bostancı, M. Sadrosadati, and O. Mutlu, "ScaleDisturb: Exploiting Temporal Asymmetry to Amplify Read Disturbance in Modern DRAM Chips," in *ISCA*, 2026.
- [173] J. Woo, J. Kim, A. Jaleel, and P. J. Nair, "Loaded Dice: Solving the Non-Selection Problem for Scalable Probabilistic RowHammer Defense," in *ISCA*, 2026.
- [174] J. Kim, S. Baek, H. Nam, M. Wi, N. S. Kim, and J. H. Ahn, "PVAC: A RowHammer Mitigation Architecture Exploiting Per-victim-row Counting," in *ISCA*, 2026.
- [175] J. Lee, M. Ahn, J. Park, and J. Jeong, "Beyond Static Policies: Dynamic-PRAC for Balanced and Efficient Rowhammer Mitigation," in *ISLPED*, 2026.
- [176] T. Fridley and O. Santos, "Mitigations Available for the DRAM Row Hammer Vulnerability," <http://blogs.cisco.com/security/mitigations-available-for-the-dram-row-hammer-vulnerability>, 2015.
- [177] Apple Inc., "About the Security Content of Mac EFI Security Update 2015-001," <https://support.apple.com/en-us/HT204934>, 2015.
- [178] Hewlett-Packard Enterprise, "HP Moonshot Component Pack Version 2015.05.0," <http://h17007.www1.hp.com/us/en/enterprise/servers/products/moonshot/component-pack/index.aspx>, 2015.
- [179] Lenovo, "Row Hammer Privilege Escalation," https://support.lenovo.com/us/en/p/roduct_security/row_hammer, 2015.
- [180] M. Ghasempour, M. Lujan, and J. Garside, "ARMOR: A Run-Time Memory Hot-Row Detector," 2015.
- [181] S. Saroiu, A. Wolman, and L. Cojocar, "The Price of Secrecy: How Hiding Internal DRAM Topologies Hurts Rowhammer Defenses," in *IRPS*, 2022.
- [182] H. Hassan, A. Olgun, A. G. Yağlıkçı, H. Luo, and O. Mutlu, "Self-Managing DRAM: A Low-Cost Framework for Enabling Autonomous and Efficient DRAM Maintenance Operations," in *MICRO*, 2024.
- [183] E. Lee, I. Kang, S. Lee, G. E. Suh, and J. H. Ahn, "TWiCe: Preventing Row-Hammering by Exploiting Time Window Counters," in *ISCA*, 2019.
- [184] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, "Counter-Based Tree Structure for Row Hammering Mitigation in DRAM," *CAL*, 2017.
- [185] O. Canpolat, A. G. Yaglikci, G. F. Oliveira, A. Olgun, O. Ergin, and O. Mutlu, "Understanding the Security Benefits and Overheads of Emerging Industry Solutions to DRAM Read Disturbance," in *DRAMSec*, 2024.
- [186] M. Qureshi, A. Rohan, G. Saileshwar, and P. J. Nair, "Hydra: Enabling Low-Overhead Mitigation of Row-Hammer at Ultra-Low Thresholds via Hybrid Tracking," in *ISCA*, 2022.
- [187] D. E. Fisch and W. C. Plants, "DRAM Adjacent Row Disturb Mitigation," 2017, US Patent 9,812,185.
- [188] Y. Wang, Y. Liu, P. Wu, and Z. Zhang, "Detect DRAM Disturbance Error by Using Disturbance Bin Counters," *CAL*, 2019.
- [189] J. M. You and J.-S. Yang, "MRLoc: Mitigating Row-Hammering Based on Memory Locality," in *DAC*, 2019.
- [190] J. Juffinger, L. Lamster, A. Kogler, M. Eichseder, M. Lipp, and D. Gruss, "CSI: Rowhammer-Cryptographic Security and Integrity Against Rowhammer," in *S&P*, 2023.
- [191] E. Manzhosov, A. Hastings, M. Pancholi, R. Piersma, M. T. I. Ziad, and S. Sethumadhavan, "Revisiting Residue Codes for Modern Memories," in *MICRO*, 2022.
- [192] F. Devaux and R. Aygnac, "Method and Circuit for Protecting a DRAM Memory Device from the Row Hammer Effect," US Patent: 10,885,966, 2021.
- [193] A. Fakhrazdehgan, Y. N. Patt, P. J. Nair, and M. K. Qureshi, "SafeGuard: Reducing the Security Risk from Row-Hammer via Low-Cost Integrity Protection," in *HPCA*, 2022.
- [194] J. Woo, G. Saileshwar, and P. J. Nair, "Scalable and Secure Row-Swap: Efficient and Safe Row Hammer Mitigation in Memory Systems," in *HPCA*, 2023.

- [195] W. Kim, C. Jung, S. Yoo, D. Hong, J. Hwang, J. Yoon, O. Jung, J. Choi, S. Hyun, M. Kang *et al.*, "A 1.1 V 16Gb DDR5 DRAM with Probabilistic-Aggressor Tracking, Refresh-Management Functionality, Per-Row Hammer Tracking, a Multi-Step Precharge, and Core-Bias Modulation for Security and Reliability Enhancement," in *ISSCC*, 2023.
- [196] Y. Wang, Y. Liu, P. Wu, and Z. Zhang, "Discreet-PARA: Rowhammer Defense with Low Cost and High Efficiency," in *ICCD*, 2021.
- [197] A. G. Yaglikci, M. Patel, J. S. Kim, R. Azizbarzoki, A. Olgun, L. Orosa, H. Hassan, J. Park, K. Kanellopoulos, T. Shahrودي, S. Ghose, and O. Mutlu, "BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows," in *HPCA*, 2021.
- [198] O. Canpolat, A. G. Yaglikci, A. Olgun, I. E. Yuksel, Y. C. Tugrul, K. Kanellopoulos, O. Ergin, and O. Mutlu, "BreakHammer: Enhancing RowHammer Mitigations by Carefully Throttling Suspect Threads," in *MICRO*, 2024.
- [199] A. G. Yaglikci, Y. C. Tugrul, G. F. De Oliveira, I. E. Yuksel, A. Olgun, H. Luo, and O. Mutlu, "Spatial Variation-Aware Read Disturbance Defenses: Experimental Analysis of Real DRAM Chips and Implications on Future Solutions," in *HPCA*, 2024.
- [200] S. Vig, S. Bhattacharya, D. Mukhopadhyay, and S.-K. Lam, "Rapid Detection of Rowhammer Attacks Using Dynamic Skewed Hash Tree," in *HASP*, 2018.
- [201] G.-H. Lee, S. Na, I. Byun, D. Min, and J. Kim, "CryoGuard: A Near Refresh-Free Robust DRAM Design for Cryogenic Computing," in *ISCA*, 2021.
- [202] A. K. Kakolyris, F. N. Bostanci, A. Olgun, I. E. Yuksel, H. Songara, K. M. Sgouras, U. Baser, K. Kanellopoulos, A. G. Yaglikci, and O. Mutlu, "ColumnKeeper: Efficient Solutions to the ColumnDisturb Vulnerability in DRAM-based Systems," in *ISCA*, 2026.
- [203] F. N. Bostanci, O. Canpolat, A. Olgun, I. E. Yuksel, K. Kanellopoulos, M. Sadrosadati, A. G. Yaglikci, and O. Mutlu, "Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses," in *MICRO*, 2025.
- [204] A. Saxena and M. Qureshi, "START: Scalable Tracking for Any Rowhammer Threshold," in *HPCA*, 2024.
- [205] A. Saxena, G. Saileshwar, J. Juffinger, A. Kogler, D. Gruss, and M. Qureshi, "PT-Guard: Integrity-Protected Page Tables to Defend Against Breakthrough Rowhammer Attacks," in *DSN*, 2023.
- [206] A. Saxena, S. Mathur, and M. Qureshi, "Rubix: Reducing the Overhead of Secure Rowhammer Mitigations via Randomized Line-to-Row Mapping," in *ASPLOS*, 2024.
- [207] A. Saxena, A. Jaleel, and M. Qureshi, "ImPress: Securing DRAM Against Data-Disturbance Errors via Implicit Row-Press Mitigation," in *MICRO*, 2024.
- [208] Z. Greenfield and T. Levy, "Throttling Support for Row-Hammer Counters," 2016, U.S. Patent 9,251,885.
- [209] Z. Greenfield, J. B. Halbert, and K. S. Bains, "Method, Apparatus and System for Determining a Count of Accesses to a Row of Memory," 2014, US Patent App. 13/626,479.
- [210] K. S. Bains and J. B. Halbert, "Row Hammer Monitoring Based on Stored Row Hammer Threshold Value," 2016, U.S. Patent 9,384,821.
- [211] S. Qazi and M. Qureshi, "DRFM and the Art of Rowhammer Sampling," in *DRAMSec*, 2025.
- [212] J. Woo and P. J. Nair, "DAPPER: A Performance-Attack-Resilient Tracker for RowHammer Defense," in *HPCA*, 2025.
- [213] J. Woo, J. Qu, G. Saileshwar, and P. J. Nair, "When Mitigations Backfire: Timing Channel Attacks and Defense for PRAC-Based RowHammer Mitigations," in *ISCA*, 2025.
- [214] Y. C. Tugrul, A. G. Yaglikci, I. E. Yuksel, A. Olgun, O. Canpolat, N. Bostanci, M. Sadrosadati, O. Ergin, and O. Mutlu, "Understanding RowHammer Under Reduced Refresh Latency: Experimental Analysis of Real DRAM Chips and Implications on Future Solutions," in *HPCA*, 2025.
- [215] H. Taneja and M. Qureshi, "From Fleet to Lab: Revisiting the Security and Complexity of Industrial Rowhammer Mitigation," arXiv:2608.26072 [cs.CR], 2026.
- [216] S. Saroiu, S. Yadalam, A. Wolman, W. Remaklus, D. Berger, I. H. Luna, I. Agarwal, and J. R. Lorch, "From Lab to Fleet: Building and Deploying a Practical Rowhammer Defense in Cloud SoCs," in *ISCA*, 2026.
- [217] K. Bains, J. Halbert, C. Mozak, T. Schoenborn, and Z. Greenfield, "Row Hammer Refresh Command," 2015, US Patent 9,117,544.
- [218] K. S. Bains and J. B. Halbert, "Distributed Row Hammer Tracking," US Patent: 9,299,400, 2016.
- [219] K. Bains *et al.*, "Method, Apparatus and System for Providing a Memory Refresh," US Patent: 9,030,903, 2015.
- [220] H. Hassan, M. Patel, J. S. Kim, A. G. Yaglikci, N. Vijaykumar, N. M. Ghiasi, S. Ghose, and O. Mutlu, "CROW: A Low-Cost Substrate for Improving DRAM Performance, Energy Efficiency, and Reliability," in *ISCA*, 2019.
- [221] S. Vittal, S. Qazi, P. Das, and M. Qureshi, "MoPAC: Efficiently Mitigating Rowhammer with Probabilistic Activation Counting," in *ISCA*, 2025.
- [222] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, "Mitigating Wordline Crosstalk Using Adaptive Trees of Counters," in *ISCA*, 2018.
- [223] Y. Park, W. Kwon, E. Lee, T. J. Ham, J. H. Ahn, and J. W. Lee, "Graphene: Strong yet Lightweight Row Hammer Protection," in *MICRO*, 2020.
- [224] M. J. Kim, J. Park, Y. Park, W. Doh, N. Kim, T. J. Ham, J. W. Lee, and J. H. Ahn, "Mithril: Cooperative Row Hammer Protection on Commodity DRAM Leveraging Managed Refresh," in *HPCA*, 2022.
- [225] D.-H. Kim, P. J. Nair, and M. K. Qureshi, "Architectural Support for Mitigating Row Hammering in DRAM Memories," *CAL*, 2014.
- [226] F. N. Bostanci, I. E. Yuksel, A. Olgun, K. Kanellopoulos, Y. C. Tugrul, A. G. Yaglikci, M. Sadrosadati, and O. Mutlu, "CoMet: Count-Min-Sketch-Based Row Tracking to Mitigate RowHammer at Low Cost," in *HPCA*, 2024.
- [227] A. Olgun, Y. C. Tugrul, N. Bostanci, I. E. Yuksel, H. Luo, S. Rhyner, A. G. Yaglikci, G. F. Oliveira, and O. Mutlu, "ABACuS: All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation," in *USENIX Security*, 2024.
- [228] G. Saileshwar, B. Wang, M. Qureshi, and P. J. Nair, "Randomized Row-Swap: Mitigating Row Hammer by Breaking Spatial Correlation Between Aggressor and Victim Rows," in *ASPLOS*, 2022.
- [229] A. Saxena, G. Saileshwar, P. J. Nair, and M. Qureshi, "AQUA: Scalable Rowhammer Mitigation by Quarantining Aggressor Rows at Runtime," in *MICRO*, 2022.
- [230] A. G. Yaglikci, A. Olgun, M. Patel, H. Luo, H. Hassan, L. Orosa, O. Ergin, and O. Mutlu, "HiRA: Hidden Row Activation for Reducing Refresh Latency of Off-the-Shelf DRAM Chips," in *MICRO*, 2022.
- [231] K. K. Chang, D. Lee, Z. Chishti, A. R. Alameldeen, C. Wilkerson, Y. Kim, and O. Mutlu, "Improving DRAM Performance by Parallelizing Refreshes with Accesses," in *HPCA*, 2014.
- [232] T. Zhang, M. Poremba, C. Xu, G. Sun, and Y. Xie, "CREAM: A Concurrent-Refresh-Aware DRAM Memory Architecture," in *HPCA*, 2014.
- [233] Z. Cui, S. A. McKee, Z. Zha, Y. Bao, and M. Chen, "DTail: A Flexible Approach to DRAM Refresh Management," in *ICS*, 2014.
- [234] Y. Riho and K. Nakazato, "Partial Access Mode: New Method for Reducing Power Consumption of Dynamic Random Access Memory," *TVLSI*, 2014.
- [235] I. Bhati, Z. Chishti, S.-L. Lu, and B. Jacob, "Flexible Auto-Refresh: Enabling Scalable and Energy-Efficient DRAM Refresh Reductions," in *ISCA*, 2015.
- [236] J. Liu, B. Jaiyen, R. Veras, and O. Mutlu, "RAIDR: Retention-Aware Intelligent DRAM Refresh," in *ISCA*, 2012.
- [237] O. Mutlu, "Retrospective: RAIDR: Retention-Aware Intelligent DRAM Refresh," *Retrospective Issue for ISCA-50*, 2023.
- [238] P. J. Nair, C.-C. Chou, and M. K. Qureshi, "Refresh Pausing in DRAM Memory Systems," *TACO*, 2014.
- [239] S. Baek, S. Cho, and R. Melhem, "Refresh Now and Then," *TC*, 2014.
- [240] A. Das, H. Hassan, and O. Mutlu, "VRL-DRAM: Improving DRAM Performance via Variable Refresh Latency," in *DAC*, 2018.
- [241] I. Bhati, Z. Chishti, and B. Jacob, "Coordinated Refresh: Energy Efficient Techniques for DRAM Refresh Scheduling," in *ISLPED*, 2013.
- [242] P. G. Emma, W. R. Reohr, and M. Meterelliyo, "Rethinking Refresh: Increasing Availability and Reducing Power in DRAM for Cache Applications," in *MICRO*, 2008.
- [243] M. Ghosh and H.-H. S. Lee, "Smart Refresh: An Enhanced Memory Controller Design for Reducing Energy in Conventional and 3D Die-stacked DRAMs," in *MICRO*, 2007.
- [244] C. Isen and L. John, "ESKIMO: Energy Savings Using Semantic Knowledge of Inconsequential Memory Occupancy for DRAM Subsystem," in *MICRO*, 2009.
- [245] J. Kim and M. C. Papaefthymiou, "Block-Based Multiperiod Dynamic Memory Design for Low Data-Retention Power," *TVLSI*, 2003.
- [246] S. Liu, K. Pattabiraman, T. Moscibroda, and B. G. Zorn, "Flicker: Saving DRAM Refresh-Power Through Critical Data Partitioning," in *ASPLOS*, 2012.
- [247] J. Mukundan, H. Hunter, K.-h. Kim, J. Stuecheli, and J. F. Martinez, "Understanding and Mitigating Refresh Overheads in High-density DDR4 DRAM Systems," in *ISCA*, 2013.
- [248] P. Nair, C.-C. Chou, and M. K. Qureshi, "A Case for Refresh Pausing in DRAM Memory Systems," in *HPCA*, 2013.
- [249] K. Patel, L. Benini, E. Macii, and M. Poncino, "Energy-Efficient Value-based Selective Refresh for Embedded DRAMs," in *PATMOS*, 2005.
- [250] J. Stuecheli, D. Kaseridis, H. C. Hunter, and L. K. John, "Elastic Refresh: Techniques to Mitigate Refresh Penalties in High Density Memory," in *MICRO*, 2010.
- [251] S. Khan, D. Lee, and O. Mutlu, "PARBOR: An Efficient System-Level Technique to Detect Data-Dependent Failures in DRAM," in *DSN*, 2016.
- [252] S. Khan, C. Wilkerson, Z. Wang, A. R. Alameldeen, D. Lee, and O. Mutlu, "Detecting and Mitigating Data-Dependent DRAM Failures by Exploiting Current Memory Content," in *MICRO*, 2017.
- [253] R. K. Venkatesan, S. Herr, and E. Rotenberg, "Retention-Aware Placement in DRAM (RAPID): Software Methods for Quasi-Non-Volatile DRAM," in *HPCA*, 2006.
- [254] S. Kim, W. Kwak, C. Kim, D. Baek, and J. Huh, "Charge-Aware DRAM Refresh Reduction with Value Transformation," in *HPCA*, 2020.
- [255] K. Nguyen, K. Lyu, X. Meng, V. Sridharan, and X. Jian, "Nonblocking Memory Refresh," in *ISCA*, 2018.
- [256] H. Kwon, K. Kim, D. Jeon, and K.-S. Chung, "Reducing Refresh Overhead with In-DRAM Error Correction Codes," in *ISQCC*, 2021.
- [257] Y. Han, Y. Wang, H. Li, and X. Li, "Data-aware DRAM Refresh to Squeeze the Margin of Retention Time in Hybrid Memory Cube," in *ICCAD*, 2014.
- [258] H. Choi, D. Hong, J. Lee, and S. Yoo, "Reducing DRAM Refresh Power Consumption by Runtime Profiling of Retention Time and Dual-Row Activation," *Microprocessors and Microsystems*, 2020.
- [259] M. Jung, E. Zulian, D. M. Mathew, M. Herrmann, C. Brugger, C. Weis, and N. Wehn, "Omitting Refresh: A Case Study for Commodity and Wide I/O DRAMs," in *MEMSYS*, 2015.
- [260] J. Kim and M. C. Papaefthymiou, "Dynamic Memory Design for Low Data-Retention Power," in *PATMOS*, 2000.
- [261] Y. Luo, S. Govindan, B. Sharma, M. Santaniello, J. Meza, A. Kansal, J. Liu, B. Khessib, K. Vaid, and O. Mutlu, "Characterizing Application Memory Error Vulnerability to Optimize Datacenter Cost via Heterogeneous-reliability Memory," in *DSN*, 2014.
- [262] S. Khan, D. Lee, Y. Kim, A. R. Alameldeen, C. Wilkerson, and O. Mutlu, "The Efficacy of Error Mitigation Techniques for DRAM Retention Failures: A Comparative Experimental Study," in *SIGMETRICS*, 2014.
- [263] I. Bhati, M.-T. Chang, Z. Chishti, S.-L. Lu, and B. Jacob, "DRAM Refresh Mechanisms, Penalties, and Trade-offs," *TC*, 2016.

- [264] A. M. Saleh, J. J. Serrano, and J. H. Patel, "Reliability of Scrubbing Recovery-Techniques for Memory Systems," *IEEE Transactions on Reliability*, 1990.
- [265] B. Schroeder, E. Pinheiro, and W.-D. Weber, "DRAM Errors in the Wild: A Large-scale Field Study," in *SIGMETRICS*, 2009.
- [266] J. Meza, Q. Wu, S. Kumar, and O. Mutlu, "Revisiting Memory Errors in Large-scale Production Data Centers: Analysis and Modeling of New Trends from the Field," in *DSN*, 2015.
- [267] V. Sridharan and D. Liberty, "A Study of DRAM Failures in the Field," in *SC*, 2012.
- [268] V. Sridharan, N. DeBardeleben, S. Blanchard, K. B. Ferreira, J. Stearley, J. Shalf, and S. Gurumurthi, "Memory Errors in Modern Systems: The Good, the Bad, and the Ugly," in *ASPLOS*, 2015.
- [269] R. Rooney and N. Koyle, "Micron DDR5 SDRAM: New Features," *Micron Technology Inc., Tech. Rep.*, 2019.
- [270] M. Patel, J. S. Kim, and O. Mutlu, "The Reach Profiler (REAPER): Enabling the Mitigation of DRAM Retention Failures via Profiling at Aggressive Conditions," in *ISCA*, 2017.
- [271] M. Patel, G. F. Oliveira, and O. Mutlu, "HARP: Practically and Effectively Identifying Uncorrectable Errors in Main Memory Chips That Use On-Die ECC," in *MICRO*, 2021.
- [272] S. Khan, C. Wilkerson, D. Lee, A. R. Alameldeen, and O. Mutlu, "A Case for Memory Content-Based Detection and Mitigation of Data-Dependent Failures in DRAM," *CAL*, 2016.
- [273] R. Sharifi and Z. Navabi, "Online Profiling for Cluster-Specific Variable Rate Refreshing in High-Density DRAM Systems," in *ETS*, 2017.
- [274] S.-L. Gong, J. Kim, S. Lym, M. Sullivan, H. David, and M. Erez, "DUO: Exposing On-Chip Redundancy to Rank-level ECC for High Reliability," in *HPCA*, 2018.
- [275] P. J. Nair, D.-H. Kim, and M. K. Qureshi, "ArchShield: Architectural Framework for Assisting DRAM Scaling by Tolerating High Error Rates," in *ISCA*, 2013.
- [276] S. Lym, H. Ha, Y. Kwon, C.-k. Chang, J. Kim, and M. Erez, "ERUCA: Efficient DRAM Resource Utilization and Resource Conflict Avoidance for Memory System Parallelism," in *HPCA*, 2018.
- [277] Y. Ryu, H. Kang, and J. Kim, "PIAT: Unleashing Memory Parallelism via Power-Interference-Aware Timing Control," *CAL*, 2026.
- [278] K. K. Chang, P. J. Nair, D. Lee, S. Ghose, M. K. Qureshi, and O. Mutlu, "Low-Cost Inter-Linked Subarrays (LISA): Enabling Fast Inter-Subarray Data Movement in DRAM," in *HPCA*, 2016.
- [279] H. Hassan, G. Pekhimenko, N. Vijaykumar, V. Seshadri, D. Lee, O. Ergin, and O. Mutlu, "ChargeCache: Reducing DRAM Latency by Exploiting Row Access Locality," in *HPCA*, 2016.
- [280] S.-L. Lu, Y.-C. Lin, and C.-L. Yang, "Improving DRAM Latency with Dynamic Asymmetric Subarray," in *MICRO*, 2015.
- [281] W. Shin, J. Yang, J. Choi, and L.-S. Kim, "NUAT: A Non-Uniform Access Time Memory Controller," in *HPCA*, 2014.
- [282] D. Lee, Y. Kim, V. Seshadri, J. Liu, L. Subramanian, and O. Mutlu, "Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture," in *HPCA*, 2013.
- [283] D. Lee, S. Khan, L. Subramanian, S. Ghose, R. Ausavarungnirun, G. Pekhimenko, V. Seshadri, and O. Mutlu, "Design-induced Latency Variation in Modern DRAM Chips: Characterization, Analysis, and Latency Reduction Mechanisms," *POMACS*, 2017.
- [284] D. Lee, Y. Kim, G. Pekhimenko, S. Khan, V. Seshadri, K. Chang, and O. Mutlu, "Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case," in *HPCA*, 2015.
- [285] Y. H. Son, O. Seongil, Y. Ro, J. W. Lee, and J. H. Ahn, "Reducing Memory Access Latency with Asymmetric DRAM Bank Organizations," in *ISCA*, 2013.
- [286] J. S. Kim, M. Patel, H. Hassan, and O. Mutlu, "Solar-DRAM: Reducing DRAM Access Latency by Exploiting the Variation in Local Bitlines," in *ICCD*, 2018.
- [287] K. Chandrasekar, S. Goossens, C. Weis, M. Koedam, B. Akesson, N. Wehn, and K. Goossens, "Exploiting Expendable Process-Margins in DRAMs for Run-Time Performance Optimization," in *DATE*, 2014.
- [288] J. Choi, W. Shin, J. Jang, J. Suh, Y. Kwon, Y. Moon, and L.-S. Kim, "Multiple Clone Row DRAM: A Low Latency and Area Optimized DRAM," in *ISCA*, 2015.
- [289] J. Lee, W. Jung, D. Kim, D. Kim, J. Lee, and J. Kim, "Agile-DRAM: Agile Trade-Offs in Memory Capacity, Latency, and Energy for Data Centers," in *HPCA*, 2024.
- [290] Y. Wang, A. Tavakkol, L. Orosa, S. Ghose, N. Mansouri Ghiasi, M. Patel, J. S. Kim, H. Hassan, M. Sadrosadati, and O. Mutlu, "Reducing DRAM Latency via Charge-Level-Aware Look-Ahead Partial Restoration," in *MICRO*, 2018.
- [291] Y. Qin, C. Lin, M. Lai, Z. Luo, S. Xu, and W. He, "Reducing DRAM Latency via In-situ Temperature- and Process-Variation-Aware Timing Detection and Adaption," in *DAC*, 2024.
- [292] H. Luo, T. Shahroodi, H. Hassan, M. Patel, A. G. Yağlıkcı, L. Orosa, J. Park, and O. Mutlu, "CLR-DRAM: A Low-Cost DRAM Architecture Enabling Dynamic Capacity-Latency Trade-Off," in *ISCA*, 2020.
- [293] S. O. Y. H. Son, N. S. Kim, and J. H. Ahn, "Row-buffer decoupling: A Case for Low-latency DRAM Microarchitecture," in *ISCA*, 2014.
- [294] Y. Wang, L. Orosa, X. Peng, Y. Guo, S. Ghose, M. Patel, J. Kim, J. Gómez-Luna, M. Sadrosadati, N. Ghiasi, and O. Mutlu, "FIGARO: Improving System Performance via Fine-Grained In-DRAM Data Relocation and Caching," in *MICRO*, 2020.
- [295] H. Du, Y. Qin, S. Chen, and Y. Kang, "FASA-DRAM: Reducing DRAM Latency with Destructive Activation and Delayed Restoration," *TACO*, 2024.
- [296] A. Olgun, F. N. Bostanci, G. F. Oliveira, Y. C. Tugrul, R. Bera, A. G. Yağlıkcı, H. Hassan, O. Ergin, and O. Mutlu, "Sectored DRAM: A Practical Energy-Efficient and High-Performance Fine-Grained DRAM Architecture," *TACO*, 2024.
- [297] T. Zhang, K. Chen, C. Xu, G. Sun, T. Wang, and Y. Xie, "Half-DRAM: A High-Bandwidth and Low-Power DRAM Architecture from the Rethinking of Fine-Grained Activation," in *ISCA*, 2014.
- [298] M. O'Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-Grained DRAM: Energy-Efficient DRAM for Extreme Bandwidth Systems," in *MICRO*, 2017.
- [299] L. Orosa, Y. Wang, M. Sadrosadati, J. S. Kim, M. Patel, I. Puddu, H. Luo, K. Razavi, J. Gómez-Luna, H. Hassan, N. Mansouri-Ghiasi, S. Ghose, and O. Mutlu, "CODIC: A Low-Cost Substrate for Enabling Custom In-DRAM Functionalities and Optimizations," in *ISCA*, 2021.
- [300] W. K. Zuravleff and T. Robinson, "Controller for a Synchronous DRAM That Maximizes Throughput by Allowing Memory Requests and Commands to Be Issued Out of Order," 1997, U.S. Patent 5,630,096.
- [301] O. Mutlu and T. Moscibroda, "Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors," in *MICRO*, 2007.
- [302] E. Ipek, O. Mutlu, J. F. Martinez, and R. Caruana, "Self-Optimizing Memory Controllers: A Reinforcement Learning Approach," in *ISCA*, 2008.
- [303] E. Ipek, O. Mutlu, J. F. Martinez, and R. Caruana, "Self-Optimizing Memory Controllers: A Reinforcement Learning Approach," *Retrospective Issue for ISCA-50*, 2023.
- [304] O. Mutlu and T. Moscibroda, "Parallelism-Aware Batch Scheduling: Enabling High-performance And Fair Shared Memory Controllers," in *ISCA*, 2008.
- [305] L. Subramanian, D. Lee, V. Seshadri, H. Rastogi, and O. Mutlu, "BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling," *TPDS*, 2016.
- [306] B. Jacob, D. Wang, and S. Ng, *Memory Systems: Cache, DRAM, Disk*. Morgan Kaufmann, 2010.
- [307] JEDEC, *JESD79F: Double Data Rate (DDR) SDRAM Standard*, 2008.
- [308] JEDEC, *JESD79-3: DDR3 SDRAM Standard*, 2012.
- [309] JEDEC, *JESD79-4C: DDR4 SDRAM Standard*, 2020.
- [310] JEDEC, *JESD79: DDR5 SDRAM Standard*, 2020.
- [311] JEDEC, *JESD79-5C: DDR5 SDRAM Standard*, 2024.
- [312] JEDEC, *JESD209-4B: Low Power Double Data Rate 4 (LPDDR4) Standard*, 2017.
- [313] JEDEC, *JESD209-5A: LPDDR5 SDRAM Standard*, 2020.
- [314] JEDEC, *JESD235C: High Bandwidth Memory (HBM) DRAM*, 2020.
- [315] JEDEC, *JESD235D: High Bandwidth Memory DRAM (HBM1, HBM2)*, 2021.
- [316] SAFARI Research Group, "Ramulator2.0 Source Code," <https://github.com/CMU-SAFARI/ramulator2>.
- [317] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A Fast and Extensible DRAM Simulator," *CAL*, 2015.
- [318] H. Luo, Y. Tugrul, F. Bostanci, A. Olgun, A. Yağlıkcı, and O. Mutlu, "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," *CAL*, 2024.
- [319] SAFARI Research Group, "Ramulator Source Code," <https://github.com/CMU-SAFARI/Ramulator>.
- [320] F. N. Bostanci, H. Luo, A. Olgun, M. Makeenkova, G. F. Oliveira, A. G. Yağlıkcı, and O. Mutlu, "Cleaning up the Mess: Re-Evaluating the Real-System Modeling Accuracy of Ramulator 2.0," in *ISPASS*, 2026.
- [321] F. N. Bostanci, H. Luo, A. Olgun, M. Makeenkova, G. F. Oliveira, A. G. Yağlıkcı, and O. Mutlu, "Extended Abstract: Re-Evaluating the Real-System Modeling Accuracy of Ramulator 2.0," in *ICS Workshops*, 2026.
- [322] H. Luo, F. N. Bostanci, A. Olgun, M. Makeenkova, Z. Malik, I. Akdeniz, and O. Mutlu, "Ramulator 2.1: A Composable Memory System Simulator for Modern DRAM Systems," in *ICS Workshops*, 2026.
- [323] SAFARI Research Group, "Ramulator# Source Code," <https://github.com/CMU-SAFARI/RamulatorSharp>.
- [324] T. Siddiqua, V. Sridharan, S. E. Raasch, N. DeBardeleben, K. B. Ferreira, S. Levy, E. Baseman, and Q. Guan, "Lifetime Memory Reliability Data from the Field," in *DFT*, 2017.
- [325] Micron, "Product Lifecycle Solutions," <https://www.micron.com/products/product-lifecycle-solutions>.
- [326] RAMExchange, "Legacy RAM Still Has Value: Here's Why," <https://www.ramexchange.net/blog/legacy-ram-still-has-value-heres-why>.
- [327] Aerospace, "Majority of Satellites Exceed Design Life," <https://aerospace.org/story/majority-satellites-exceed-design-life>.
- [328] G. Fox, R. Salazar, H. Habib-Agahi, and G. F. Dubos, "A Satellite Mortality Study to Support Space Systems Lifetime Prediction," in *Aerospace*, 2013.
- [329] NASA, "Curiosity Rover Updates," <https://science.nasa.gov/mission/msl-curiosity/science-updates/>.
- [330] Spaceflight Now, "Curiosity Rover's Computer Built for the Rigors of Mars," <https://www.spaceflightnow.com/mars/msl/120810computer/>.
- [331] R. Berger, D. Bayles, R. Brown, S. Doyle, A. Kazemzadeh, K. Knowles, D. Moser, J. Rodgers, B. Saari, D. Stanley, and B. Grant, "The RAD750™ - A Radiation Hardened PowerPC™ Processor for High Performance Spaceborne Applications," in *Aerospace*, 2001.
- [332] 3D PLUS, "Radiation Tolerant SDRAM Space Memory," <https://www.3d-plus.com/products/space-radiation-tolerant-sdram/>.
- [333] 3D PLUS, "Radiation Tolerant DDR2 SDRAM Space Memory," <https://www.3d-plus.com/products/space-radiation-tolerant-ddr2-sdram/>.
- [334] 3D PLUS, "Radiation Tolerant DDR3 SDRAM Space Qualified Memory," <https://www.3d-plus.com/products/space-radiation-tolerant-ddr3-sdram/>.
- [335] NASA, "Parker Solar Probe," <https://science.nasa.gov/blogs/parker-solar-probe/>.
- [336] European Space Agency, "BepiColombo," https://www.esa.int/Science_Exploration/Space_Science/BepiColombo.
- [337] NASA, "Mars 2020: Perseverance Rover," <https://science.nasa.gov/mission/mars-2020-perseverance/>.
- [338] 3D PLUS, "EMIT," <https://www.3d-plus.com/heritages/emit/>.
- [339] NASA, "Voyager," <https://science.nasa.gov/mission/voyager/>.

- [340] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. Patterson, "Ten Lessons From Three Generations Shaped Google's TPUv4: Industrial Product," in *ISCA*, 2021.
- [341] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings," in *ISCA*, 2023.
- [342] P. Jattke, V. van der Veen, P. Frigo, S. Gunter, and K. Razavi, "Blacksmith: Scalable Rowhammering in the Frequency Domain," in *S&P*, 2022.
- [343] K. Chandrasekar, B. Akesson, and K. Goossens, "Improved Power Modeling of DDR SDRAMs," in *DSD*, 2011.
- [344] Microelectronic Systems Design Research Group, "DRAMPower," <https://github.com/tukl-msd/DRAMPower>.
- [345] T. O. Project, "OpenROAD Flow," <https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git>.
- [346] NanGate, "NanGate FreePDK45 Open Cell Library," <https://tilos-ai-institute.github.io/MacroPlacement/Enablers/NanGate45/>.
- [347] S. Sarangi and B. Baas, "DeepScaleTool: A Tool for the Accurate Estimation of Technology Scaling in the Deep-Submicron Era," in *ISCA*, 2021.
- [348] Intel, "Emerald Rapids - Intel," <https://www.intel.com/content/www/us/en/products/sku/237252/intel-xeon-platinum-8593q-processor-320m-cache-2-20-ghz/specifications.html>, 2024.
- [349] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory Access Scheduling," in *ISCA*, 2000.
- [350] Y. Kim, D. Han, O. Mutlu, and M. Harchol-Balter, "ATLAS: A Scalable and High-Performance Scheduling Algorithm for Multiple Memory Controllers," in *HPCA*, 2010.
- [351] C. J. Lee, V. Narasiman, E. Ebrahimi, O. Mutlu, and Y. N. Patt, "DRAM-Aware Last-Level Cache Writeback: Reducing Write-Caused Interference in Memory Systems," *HPS Technical Report*, 2010.
- [352] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "HyCUBE: A CGRA with Reconfigurable Single-Cycle Multi-Hop Interconnect," in *DAC*, 2017.
- [353] B. Wang, M. Karunaratne, A. Kulkarni, T. Mitra, and L.-S. Peh, "HyCUBE: A 0.9V 26.4 MOPS/mW, 290 pJ/op, Power Efficient Accelerator for IoT Applications," in *A-SSCC*, 2019.
- [354] C. Kim, M. Chung, Y. Cho, M. Konijnenburg, S. Ryu, and J. Kim, "ULP-SRP: Ultra Low Power Samsung Reconfigurable Processor for Biomedical Applications," in *FPT*, 2012.
- [355] P. Whatmough, S. Lee, S. Xi, U. Gupta, L. Pentecost, M. Donato, H. Hseuh, D. Brooks, and G. Wei, "SMIV: A 16nm SoC with Efficient and Flexible DNN Acceleration for Intelligent IoT Devices," *HCS*, 2018.
- [356] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A Reconfigurable Architecture for Parallel Patterns," in *ISCA*, 2017.
- [357] C. Kumar, A. Seshadri, A. Chaudhary, S. Bhawalkar, R. Singh, and E. Rotenberg, "Post-Fabrication Microarchitecture," in *MICRO*, 2021.
- [358] A. Li, A. Ning, and D. Wentzlaff, "Duet: Creating Harmony between Processors and Embedded FPGAs," in *HPCA*, 2023.
- [359] UC Berkeley Architecture Research, "riscv-mini," <https://github.com/ucb-bar/riscv-mini>.
- [360] UEFI Forum, *UEFI Platform Initialization Specification Version 1.9*, https://uefi.org/sites/default/files/resources/UEFI_PI_Spec_Final_Draft_1.9.pdf, 2024.
- [361] Intel, *Intel 700 Series Chipset Family On-Package Platform Controller Hub (PCH) Datasheet*, 2023.
- [362] Intel, *Intel Architecture Software Developer's Manual*, 2018.
- [363] D. A. Cooper, W. T. Polk, A. R. Regenscheid, and M. P. Souppaya, *SP 800-147. BIOS Protection Guidelines*, 2011.
- [364] Intel, *Second Generation Intel® Xeon® Scalable Processors*, 2019.
- [365] B. Keeth, R. J. Baker, B. Johnson, and F. Lin, *DRAM Circuit Design: Fundamental and High-Speed Topics*. John Wiley & Sons, 2007.
- [366] Standard Performance Evaluation Corporation, "SPEC 2006," <http://www.spec.org/cpu2006/>, 2006.
- [367] Standard Performance Evaluation Corporation, "SPEC 2017," <http://www.spec.org/cpu2017/>, 2017.
- [368] Transaction Processing Performance Council, "TPC Benchmarks," <http://tpc.org/>.
- [369] J. E. Fritts, F. W. Steiling, J. A. Tucek, and W. Wolf, "MediaBench II Video: Expediting the Next Generation of Video Systems Research," *MICPRO*, 2009.
- [370] B. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking Cloud Serving Systems with YCSB," in *SoCC*, 2010.
- [371] A. Snaveily and D. M. Tullsen, "Symbiotic Job Scheduling for A Simultaneous Multithreaded Processor," in *ASPLOS*, 2000.
- [372] S. Eyerhan and L. Eeckhout, "System-Level Performance Metrics for Multiprogram Workloads," *IEEE Micro*, 2008.
- [373] P. Michaud, "Demystifying Multicore Throughput Metrics," *CAL*, 2012.
- [374] G. F. Oliveira, J. Gómez-Luna, L. Orosa, S. Ghose, N. Vijaykumar, I. Fernandez, M. Sadosadati, and O. Mutlu, "DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks," in *IEEE Access*, 2021.
- [375] SAFARI Research Group, "PiDRAM Source Code," <https://github.com/CMU-SAFARI/PiDRAM>.
- [376] L. Cojocar, J. Kim, M. Patel, L. Tsai, S. Saroiu, A. Wolman, and O. Mutlu, "Are We Susceptible to Rowhammer? An End-to-End Methodology for Cloud Providers," *S&P*, 2020.
- [377] SAFARI Research Group, "Chronus Source Code," <https://github.com/CMU-SAFARI/Chronus>.
- [378] B. R. Rau, "Pseudo-Randomly Interleaved Memory," in *ISCA*, 1991.
- [379] K. K. Chang, A. Kashyap, H. Hassan, S. Ghose, K. Hsieh, D. Lee, T. Li, G. Pekhimenko, S. Khan, and O. Mutlu, "Understanding Latency Variation in Modern DRAM Chips: Experimental Characterization, Analysis, and Optimization," in *SIGMETRICS*, 2016.
- [380] Embedded Computing Lab, "Morpher: An Open-Source Tool for CGRA Accelerators," <https://github.com/ecolab-nus/morpher>.
- [381] R. Juneja, P. Dangi, T. K. Bandara, Z. Li, D. Wijerathne, L.-S. Peh, and T. Mitra, "Building an Open CGRA Ecosystem for Agile Innovation," arXiv:2508.19090 [cs.AR], 2025.
- [382] A. Farmahini-Farahani, J. H. Ahn, K. Morrow, and N. S. Kim, "NDA: Near-DRAM Acceleration Architecture Leveraging Commodity DRAM Devices and Standard Memory Modules," in *HPCA*, 2015.
- [383] F. Devaux, "The True Processing In Memory Accelerator," in *HCS*, 2019.
- [384] L. Ke, X. Zhang, J. So, J.-G. Lee, S.-H. Kang, S. Lee, S. Han, Y. Cho, J. H. Kim, Y. Kwon, K. Kim, J. Jung, I. Yun, S. J. Park, H. Park, J. Song, J. Cho, K. Sohn, N. S. Kim, and H.-H. S. Lee, "Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM," *IEEE Micro*, 2022.
- [385] M. Gao and C. Kozyrakis, "HRL: Efficient and Flexible Reconfigurable Logic for Near-Data Processing," in *HPCA*, 2016.
- [386] O. O. Babarinsa and S. Idreos, "JAFAR: Near-Data Processing for Databases," in *SIGMOD*, 2015.
- [387] B. Yoon, S. Han, G. Park, and J.-J. Kim, "SplitSync: Bank Group-Level Split-Synchronization for High-Performance DRAM PIM," in *DAC*, 2025.
- [388] T. Moscibroda and O. Mutlu, "Distributed Order Scheduling and its Application to Multi-Core DRAM Controllers," in *PODC*, 2008.
- [389] H. Nam, S. Baek, M. Wi, M. J. Kim, J. Park, C. Song, N. S. Kim, and J. H. Ahn, "DRAMScope: Uncovering DRAM Microarchitecture and Characteristics by Issuing Memory Commands," in *ISCA*, 2024.
- [390] H. Nam, S. Baek, M. Wi, M. J. Kim, J. Park, C. Song, N. S. Kim, and J. H. Ahn, "X-ray: Discovering DRAM Internal Structure and Error Characteristics by Issuing Memory Commands," *IEEE CAL*, 2023.
- [391] H. Luo, İsmail Emir Yüksel, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Revisiting DRAM Read Disturbance: Identifying Inconsistencies Between Experimental Characterization and Device-Level Studies," in *VTS*, 2025.
- [392] A. Olgun, F. N. Bostanci, I. E. Yüksel, H. Luo, M. Patel, A. G. Yağlıkçı, and O. Mutlu, "DiscoRD: An Experimental Methodology for Quickly Discovering the Reliable Read Disturbance Threshold of Real DRAM Chips," arXiv:2603.12435 [cs.AR], 2026.
- [393] A. Olgun, F. N. Bostanci, I. E. Yüksel, H. Luo, M. Patel, A. G. Yağlıkçı, and O. Mutlu, "QRD: An Empirical Framework for Quantifying DRAM Read-Disturbance-Induced System Failure Probability," in *ICCD*, 2026.
- [394] M. Hashemi, Khubaib, E. Ebrahimi, O. Mutlu, and Y. N. Patt, "Accelerating Dependent Cache Misses with an Enhanced Memory Controller," in *ISCA*, 2016.
- [395] A. Khadem, K. Kamalakannan, Z. Zhu, A. Poptani, Y. Gu, J. B. Dominguez-Trujillo, N. Talati, D. Fujiki, S. Mahlke, G. Shipman, and R. Das, "DX100: Programmable Data Access Accelerator for Indirection," in *ISCA*, 2025.
- [396] K. Hsieh, E. Ebrahimi, G. Kim, N. Chatterjee, M. O'Connor, N. Vijaykumar, O. Mutlu, and S. W. Keckler, "Transparent Offloading and Mapping (TOM): Enabling Programmer-Transparent Near-Data Processing in GPU Systems," in *ISCA*, 2016.
- [397] A. Boroumand, S. Ghose, M. Patel, H. Hassan, B. Lucia, R. Ausavarungnirun, K. Hsieh, N. Hajinazar, K. T. Malladi, H. Zheng, and O. Mutlu, "CoNDA: Efficient Cache Coherence Support for Near-Data Accelerators," in *ISCA*, 2019.
- [398] A. Boroumand, S. Ghose, B. Lucia, K. Hsieh, K. Malladi, H. Zheng, and O. Mutlu, "LazyPIM: An Efficient Cache Coherence Mechanism for Processing-in-Memory," *CAL*, 2017.
- [399] S. K. Vittal and M. Qureshi, "BARD: Reducing Write Latency of DDR5 Memory by Exploiting Bank-Parallelism," in *HPCA*, 2026.
- [400] M. Hashemi, O. Mutlu, and Y. N. Patt, "Continuous Runahead: Transparent Hardware Acceleration for Memory Intensive Workloads," in *MICRO*, 2016.
- [401] H. Hassan, A. Olgun, A. G. Yağlıkçı, H. Luo, and O. Mutlu, "Self-Managing DRAM: A Low-Cost Framework for Enabling Autonomous and Efficient in-DRAM Operations," arXiv:2207.13358 [cs.AR], 2025.
- [402] M. N. Bojnordi and E. Ipek, "PARDIS: A Programmable Memory Controller for the DDRx Interfacing Standards," in *ISCA*, 2012.
- [403] SAFARI Research Group, "DRAM Bender — GitHub Repository," <https://github.com/CMU-SAFARI/DRAM-Bender>, 2022.
- [404] A. Olgun, H. Hassan, A. G. Yağlıkçı, Y. C. Tuğrul, L. Orosa, H. Luo, M. Patel, O. Ergin, and O. Mutlu, "DRAM Bender: An Extensible and Versatile FPGA-based Infrastructure to Easily Test State-of-the-art DRAM Chips," *TCAD*, 2023.
- [405] A. Olgun, H. Luo, I. E. Yüksel, F. N. Bostanci, A. G. Yağlıkçı, and O. Mutlu, "A Modern Large-Scale Memory Characterization Laboratory," in *ICS Workshops*, 2026.
- [406] SAFARI Research Group, "SoftMC Source Code," <https://github.com/CMU-SAFARI/SoftMC>.
- [407] H. Hassan, N. Vijaykumar, S. Khan, S. Ghose, K. Chang, G. Pekhimenko, D. Lee, O. Ergin, and O. Mutlu, "SoftMC: A Flexible and Practical Open-Source Infrastructure for Enabling Experimental DRAM Studies," in *HPCA*, 2017.
- [408] M. Patel, J. Kim, T. Shahroodi, H. Hassan, and O. Mutlu, "Bit-Exact ECC Recovery (BEER): Determining DRAM On-Die ECC Functions by Exploiting DRAM Data Retention Characteristics," in *MICRO*, 2020.
- [409] K. K. Chang, A. G. Yağlıkçı, S. Ghose, A. Agrawal, N. Chatterjee, A. Kashyap, D. Lee, M. O'Connor, H. Hassan, and O. Mutlu, "Understanding Reduced-Voltage Operation in Modern DRAM Devices: Experimental Characterization, Analysis, and Mechanisms," in *SIGMETRICS*, 2017.

- [410] I. E. Yuksel, A. Olgun, F. N. Bostanci, O. Canpolat, G. F. Oliveira, M. Sadrosadati, A. G. Yaglikci, and O. Mutlu, "In-DRAM True Random Number Generation Using Simultaneous Multiple-Row Activation: An Experimental Study of Real DRAM Chips," in *ICCD*, 2025.
- [411] O. Canpolat, A. Olgun, D. Novo, O. Ergin, and O. Mutlu, "EasyDRAM: An FPGA-based Infrastructure for Fast and Accurate End-to-End Evaluation of Emerging DRAM Techniques," in *DSN*, 2025.
- [412] SAFARI Research Group, "EasyDRAM Source Code," <https://github.com/CMU-SAFARI/EasyDRAM>.
- [413] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, "Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning," in *MICRO*, 2021.
- [414] R. Bera, K. Kanellopoulos, S. Balachandran, D. Novo, A. Olgun, M. Sadrosadati, and O. Mutlu, "Hermes: Accelerating Long-Latency Load Requests via Perceptron-Based Off-Chip Load Prediction," in *MICRO*, 2022.
- [415] K. Kanellopoulos, K. Sgouras, H. Songara, A. K. Kakolyris, V.-P. Nitu, S. Galanopoulos, R. Bera, K. Koliogeorgi, R. Kumar, and O. Mutlu, "Revelator: Rapid Data Fetching via System-Software-Guided Hash-based Speculative Address Translation," in *ISCA*, 2026.
- [416] R. Bera, R. Nadig, and O. Mutlu, "Machine-Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage," *IEEE Micro*, 2026.

A. Modifications Required by the Analyzed DRAM Techniques

This appendix discusses the DRAM system modifications required by the 32 techniques in Table 1. Table 3 classifies these modifications into three categories: memory controller (MC), DRAM interface (IF), or DRAM chip (Chip) modifications. We define these categories below.

Memory controller modification. A technique requires a memory controller modification if a conventional memory controller that follows the DRAM standard cannot support it. Such modifications typically include adding memory controller state (e.g., activation counters or timers), conditions to trigger techniques, or rules for updating controller state. The controller modifications may also be required to issue new DRAM commands or use existing DRAM commands with timing constraints that the standard does not specify.

DRAM interface modification. A technique requires a

Table 3: Modifications required by each of the 32 analyzed DRAM techniques. MC = memory controller, IF = DRAM interface, Chip = DRAM chip.

Technique	MC	IF	Chip
<i>Processing-using-DRAM</i>			
RowClone [34]	✓	✓	✓
Ambit [35]	✓	×	✓
ComputeDRAM [81]	✓	×	×
FracDRAM [82]	✓	×	×
PRADA [104]	✓	✓	✓
<i>Probabilistic controller-side read disturbance mitigation</i>			
PARA [36]	✓	×	×
MoPAC-C [221]	✓	✓	×
<i>Subarray-level parallelism</i>			
MASA [48]	✓	✓	✓
SARP [231]	✓	×	✓
HiRA [230]	✓	×	×
<i>Access latency reduction</i>			
TL-DRAM [282]	✓	×	✓
NUAT [281]	✓	×	×
DAS-DRAM [280]	✓	×	✓
ChargeCache [279]	✓	×	×
LISA [278]	✓	✓	✓
CROW [220]	✓	✓	✓
FIGARO [294]	✓	✓	✓
FASA-DRAM [295]	✓	✓	✓
<i>Deterministic controller-side read disturbance mitigation</i>			
Counter-based [36]	✓	×	×
CRA [225]	✓	×	×
CAT [222]	✓	×	×
Graphene [223]	✓	✓	×
Mithril [224]	✓	×	✓
AQUA [229]	✓	×	×
RRS [228]	✓	×	×
ABACuS [227]	✓	×	×
CoMeT [226]	✓	×	×
<i>In-DRAM true random number generation</i>			
D-RaNGe [80]	✓	×	×
QUAC-TRNG [83]	✓	×	×
<i>Fine-grained activation</i>			
Half-DRAM [297]	✓	×	✓
FGDRAM [298]	✓	✓	✓
Sectorized DRAM [296]	✓	✓	✓
Total (of 32)	32	11	15

DRAM interface modification if it introduces a DRAM command that the DRAM standard does not define or assigns a new functional meaning to bits that the standard reserves or leaves unused. This definition excludes techniques that issue only standard DRAM commands, even if they issue them in sequences or with timing constraints that the standard does not specify, or if a modified DRAM chip responds to them differently.

DRAM chip modification. A technique requires a DRAM chip modification if it proposes to change the DRAM cell array, its peripheral circuits, or both. A technique that works on unmodified, commercially available DRAM chips does *not* require a DRAM chip modification.

B. Artifact Appendix

B.1. Abstract

Our artifact provides the source code, scripts, and data required to reproduce the results of Terracotta.

B.1.1. Source code, scripts, and data. We provide (i) a Ramulator 2.0-based cycle-level simulator integrated with DRAM-Power and extended to model custom and Terracotta implementations of PRADA [104,105], MoPAC-C [221], MASA [48], ChargeCache [279], and the composition of ChargeCache and MASA; (ii) RTL sources for evaluating controller area and power; (iii) workload memory traces; and (iv) Bash and Python scripts to build the tools, run experiments, parse results, and generate plots.

B.1.2. Reproducible results. The artifact supports reproduction of the performance and DRAM energy results (Fig. 9–Fig. 16), memory access latency results (§8.6.2), and controller area and power results (§8.6.3).

B.2. Artifact check-list (meta-information)

- **Program:** C++ and RTL programs; Python 3 and Bash scripts.
- **Compilation:** C++20 toolchain (tested with GCC 11.4.0+) and CMake (tested with 3.22.1).
- **Benchmarks:** 57 workload traces from SPEC CPU2006 [366], SPEC CPU2017 [367], TPC [368], MediaBench [369], and YCSB [370]; five PRADA microbenchmarks.
- **Runtime environment:** Linux (Ubuntu 22.04), Python 3.10.6, Slurm 21.08.5, and Podman 4.5.1 or Docker 24.0.5. Slurm is needed only for cluster runs.
- **Hardware:** x86-64 system; 32 GiB RAM for building and hardware evaluation; up to 4 GiB RAM per simulation. A cluster with at least 1000 cores is recommended for the full sweep.
- **Metrics:** IPC, weighted speedup, DRAM energy, memory access latency, and area and power.
- **Outputs:** PDF/PNG figures and text reports reproducing Fig. 9–Fig. 16, §8.6.2, and §8.6.3.
- **Experiments:** (E1) performance and DRAM energy; (E2) hardware complexity.
- **How much disk space is required (approximately)?:** 80 GiB.
- **How much time is needed to prepare the workflow (approximately)?:** 75 minutes, largely unattended.
- **How much time is needed to complete experiments (approximately)?:** One day on a 1000-core cluster.

- **Publicly available:** Yes.
- **Automation:** Bash and Python scripts.
- **Archived:** Source DOI: [10.5281/zenodo.21535719](https://doi.org/10.5281/zenodo.21535719); traces DOI: [10.5281/zenodo.21541650](https://doi.org/10.5281/zenodo.21541650).

B.3. Description

B.3.1. Key results (R).

- (R1) Performance of four DRAM techniques and the composition of ChargeCache and MASA (Fig. 9–Fig. 13).
- (R2) DRAM energy overheads (Fig. 14).
- (R3) 99th-percentile memory access latency (§8.6.2).
- (R4) Sensitivity to controller latency overhead (Fig. 15).
- (R5) Controller area and power overheads over custom implementations (§8.6.3).
- (R6) Performance of Terracotta using programmable tables or reconfigurable logic (Fig. 16).

B.3.2. Experiment workflows (E).

- (E1) Performance and DRAM energy evaluation to reproduce R1–R4 and R6.
- (E2) Hardware complexity evaluation to reproduce R5.

B.3.3. How to access. The source code and scripts are available at <https://github.com/CMU-SAFARI/Terracotta> and traces are available on Zenodo at <https://doi.org/10.5281/zenodo.21541650>.

B.3.4. Hardware dependencies. The artifact runs on x86-64 systems with the following resources:

1. At least 32 GiB RAM for building the artifact and evaluating hardware complexity.
 2. Approximately 80 GiB of free disk space for source code, data, and results.
 3. Up to 4 GiB RAM and one core per simulation job.
- The full sweep comprises approximately 5000 jobs. We recommend a cluster with at least 1000 cores to complete it in approximately one day.

B.3.5. Software dependencies.

1. Linux (tested on Ubuntu 22.04).
2. CMake (tested with 3.22.1).
3. C++20 toolchain (tested with GCC 11.4.0+).
4. Python 3.10, pip3, and the packages matplotlib, pandas, and pyyaml.
5. Git, wget, tar, and unzip.
6. Slurm 21.08.5 for cluster runs.
7. Podman 4.5.1 or Docker 24.0.5 for containerized runs; required for E2 and optional for E1.

The artifact’s README.md provides installation instructions for common dependencies.

B.3.6. Workload memory traces. We use workload memory traces collected from the SPEC CPU2006 [366], SPEC CPU2017 [367], TPC [368], MediaBench [369], and YCSB [370] suites. For PRADA, five microbenchmarks cover COPY, NOT, AND/OR, NAND/NOR, and XOR/XNOR computation operations in one bank and across 32 banks (§8.1).

B.4. Installation

Clone the repository, then run the remaining commands from its root directory.

1. Obtain the source code.

```
git clone \
  https://github.com/CMU-SAFARI/Terracotta.git
cd Terracotta
git submodule update --init --recursive \
  RTL/OpenROAD-flow-scripts
```

2. Install Python dependencies.

```
python3 -m pip install -r requirements.txt
```

3. Download the traces.

```
python3 download_traces.py
```

4. Build the tools. For E1, build Ramulator 2.0 on the host. We also support containerized builds. Please see the artifact’s README.md for details.

```
./ramulator2/build_ramulator2_release.sh
```

For E2, build the OpenROAD and Yosys container image. We only support a containerized build for E2.

```
./build_hw.sh
```

B.5. Experiment workflow

E1: Performance and DRAM energy workflow.

1. Launch the sweep. We recommend using a Slurm cluster. The launcher throttles the number of simulation jobs (default 700), so please run the experiment launcher in detached mode so that experiments can continue after logout.

```
nohup ./run_with_slurm.sh all > sweep.log 2>&1 &
```

Alternatively, on a personal computer:

```
nohup ./run_with_personalcomputer.sh --no-container \
  all > sweep.log 2>&1 &
```

2. Track simulation progress. Due to the large number of jobs, we provide scripts that allow progress tracking.

```
./check_progress.sh --watch # live dashboard
./check_run_status.sh all # completion status
```

3. Parse and plot. When all simulations are complete, run the following scripts to parse the results and reproduce the paper’s key results.

```
./parse_results.sh
./plot_all_figures.sh
```

Plots are written to figures/, CSV data to results/csv/, and text reports to results/txt/.

E2: Hardware complexity workflow.

Run synthesis. This workflow can run on a personal computer:

```
nohup ./run_hw_with_personalcomputer.sh \
  > hw.log 2>&1 &
```

On a Slurm cluster:

```
nohup ./run_hw_with_slurm.sh > hw.log 2>&1 &
```

The CSV outputs are `synthesis_results.csv` and `terraccotta_synthesis_results.csv` under `RTL/results/`. The summary is `RTL/results/txt/hw_complexity.txt`.

B.6. Evaluation and expected results

E1: Performance and DRAM energy.

E1 produces plots for R1, R2, R4, and R6 in `figures/`, in both PDF and PNG formats, and a text report for R3.

(R1) Performance. Plots `figure9`–`figure13` reproduce Fig. 9–Fig. 13. They show that Terracotta retains almost all of the performance benefits of custom implementations.

(R2) DRAM energy. Plot `figure14` reproduces Fig. 14 and shows that Terracotta incurs low DRAM energy overheads relative to custom implementations.

(R3) Memory access latency. `results/txt/latency_comparison.txt` reports the 99th-percentile memory access latency results in §8.6.2, showing a marginal 0.17% increase over custom implementations.

(R4) Controller latency sensitivity. Plot `figure15` reproduces Fig. 15 and shows the sensitivity of Terracotta to controller latency overhead.

(R6) Programmable tables and reconfigurable logic. Plot `figure16` reproduces Fig. 16 and compares the performance of programmable table-based and CGRA-based Terracotta im-

plementations. Details on deriving the CGRA latencies are in `cgra/README.md`.

E2: Hardware complexity.

E2 reproduces the controller area and power results (R5).

(R5) Controller area and power. `RTL/results/txt/hw_complexity.txt` reports the results in §8.6.3. The additional controller area and power over custom implementations amount to 0.03% of die area and 0.56% of TDP of the reference Intel Xeon processor [348].

B.7. Experiment customization

The scripts support customization of (i) the evaluated techniques, (ii) simulation parameters (e.g., number of instructions, number of concurrent jobs), (iii) the runtime environment (e.g., Slurm vs. personal computer), (iv) the workloads and mixes, and (v) containerization (e.g., Podman vs. Docker). Details on how to customize the experiments are provided in the artifact’s `README.md`.

B.8. Methodology

Submission, reviewing, and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>